

Herramienta integral de análisis de sentimientos en redes sociales

Trabajo final de la carrera de grado de
Ingeniería de Sistemas de la
Universidad Nacional del Centro de la Prov. de Bs.As.
Por Eric Román Mastrángelo



Directores: Dr. Javier A. Dottori

Dr. Marcelo G. Armentano

Tandil, Argentina, Agosto 2018

Agradecimientos

Quiero agradecer a todas las personas que han posibilitado la realización de este trabajo, tanto directa como indirectamente. A los directores de este proyecto, Javier Dottori y Marcelo Armentano que me han ayudado brindándome su apoyo y conocimiento a lo largo del mismo.

Gracias a Eliana Giunta y Analía Fatta por haberse tomado la molestia de entrenarme en algunas de las tecnologías con las que desarrollé este proyecto.

A mis amigos y grupos de estudio, que me acompañaron durante el transcurso de toda la carrera y de quienes me llevo grandes aprendizajes.

Quiero agradecer a toda mi familia, quienes me dieron su apoyo siempre. Todo el esfuerzo dedicado en este trabajo y a lo largo de los años de carrera va dedicado a todos ellos.

Eric Román Mastrángelo

Índice General

Resumen	8
Capítulo 1	
Introducción	9
1.1 Motivación	10
1.2 Objetivos de la Tesis	12
Capítulo 2	
Marco Teórico	14
2.1 Análisis de redes sociales	14
2.2 Análisis de Sentimientos	15
2.3 Algoritmos de clasificación utilizados	16
2.3.1 Stanford CoreNLP	16
2.3.2 LingPipe	19
2.4 REST - Servicios Web	20
2.5 WebSocket	22
2.6 Alternativas para el sistema	23
Capítulo 3	
Solución propuesta	26
3.1 Relevamiento de requerimientos	26
3.1.1 Obtención de Información	27
3.1.2 Análisis de la información	29
3.1.3 Visualización de los resultados	30
3.2 Aplicación propuesta	30
Capítulo 4	
Diseño e implementación	33
4.1 Tecnologías de desarrollo	33

4.1.1 Cliente	34
4.1.2 Servidor	35
4.1.3 Herramienta de BI	35
4.1.4 Bibliotecas externas	37
4.2 Etapas del proyecto	38
4.2.1 Etapa 1: Prototipo con componentes aislados	38
4.2.1.1 Twitter API	38
4.2.1.2 Qlik Sense	41
4.2.1.3 Instanciación de los algoritmos de clasificación	43
4.2.2 Etapa 2: Desarrollo Web	45
4.2.2.1 Front end	45
4.2.2.2 Back end	47
4.2.3 Etapa 3: Integración Qlik Web	50
4.3 Arquitectura final del sistema	53
4.3.1 Vistas estáticas del Sistema	53
Servidor	53
Cliente	56
4.3.2 Vistas dinámicas del Sistema	57
Capítulo 5	
Metodología	60
5.1 Algoritmos utilizados	60
5.1.1 Stanford CoreNLP	60
5.1.2 LingPipe	61
5.2 Base de Datos de entrenamiento y validación	63
5.3 Métricas de Algoritmos	65
5.4 Preprocesamiento de tweets	67

Capítulo 6	
Resultados	71
6.1 Métricas de Algoritmos	71
6.1.1 Precisión	71
6.1.2 Recall	73
6.1.3 Performance	74
6.1.4 Confianza	75
6.1.5 Análisis de resultados	76
6.2 Caso de estudio 1: RespuestasTweet	78
6.2.1 Usuario estudiado 1: Demi Lovato	78
6.2.2 Usuario estudiado 2: Donald Trump	80
6.3 Caso de estudio 2: Hashtag	84
6.4 Caso de estudio 3: TweetsUsuario	87
Capítulo 7	
Conclusiones	90
7.1 Limitaciones y trabajos futuros	92
Bibliografía	93
Anexo 1	
Pseudocódigos	96
Algoritmo 1	96
Algoritmo 2	96
Algoritmo 3	97
Anexo 2	
Aplicación Web	97

Índice de figuras

- 1.1 Crecimiento en consumo de redes sociales.
- 2.1 Ejemplo de árbol sintáctico.
- 2.2 Flujo de clasificación.
- 2.3 Costos herramienta paga alternativa.
- 3.1 Modelo Cliente-Servidor.
- 3.2 Arquitectura inicial del sistema.
- 4.1 Qlik Sense Demo.
- 4.2 Claves y tokens de acceso para la API de Twitter.
- 4.3 Diagrama de clases patrón Strategy.
- 4.4 Conexión rest de aplicación Qlik Sense.
- 4.5 Añadir datos a un gráfico de Qlik Sense.
- 4.6 Diagrama de clases de algoritmos, patrón Strategy.
- 4.7 Componente intermedio entre cliente-servidor.
- 4.8 Interfaz de usuario.
- 4.9 Diagrama de componentes del servidor.
- 4.10 Patrón Adapter Cliente-Servidor.
- 4.11 Diagrama de paquetes del servidor.
- 4.12 Diagrama de componentes del cliente.
- 4.13 Arquitectura cliente-servidor de la aplicación.
- 4.14 Diagrama de actividades de la aplicación.
- 4.15 Diagrama de secuencia Cliente-Servidor.
- 5.1 Esquema de ubicación de datos de entrenamiento.
- 5.2 Ejemplo de un tweet.
- 6.1 Precisión Algoritmos.
- 6.2 Recall Algoritmos.
- 6.3 Distribución de confianza Algoritmos.
- 6.4 Exactitud para cada valor de confianza.

- 6.5 Resultado de la clasificación de respuestas Usuario 1.
- 6.6 Respuestas negativas Usuario 1.
- 6.7 Resultado de la clasificación de respuestas Usuario 2.
- 6.8 Resultados filtrados a positivos.
- 6.9 Resultados filtrados a neutros.
- 6.10 Resultados filtrados a negativos.
- 6.11 Obtener Hashtag.
- 6.12 Resultado de Clasificar tweets con hashtag.
- 6.13 Suma de retweets para cada categoría.
- 6.14 Tweet más retwiteado con #TrumpMustGo.
- 6.15 Resultado de Clasificar tweets de McDonald's, Coca Cola y Ubuntu.
- 6.16 Total de retweets para McDonald's, Coca Cola y Ubuntu.

Índice de tablas

- 3.1 Tabla comparativa redes sociales.
- 3.2 Tabla comparativa aplicación de escritorio versus web.
- 5.1 Tabla comparativa bases de datos encontradas.
- 5.2 Tabla de elementos reconocidos y extraídos de un tweet.
- 5.3 Investigación de tipos de Hashtags y su frecuencia.
- 6.1 Performance algoritmos.
- 6.2 Tabla comparativa Precision y Recall.
- 6.3 Tabla comparativa confianza

Resumen

La identificación del sentimiento en un texto es un campo de estudio importante (opinion mining-sentiment analysis), con plataformas de medios sociales como Twitter que atraen el interés de los investigadores en el procesamiento del lenguaje, así como en las ciencias políticas y sociales. La tarea generalmente implica detectar si un texto expresa un sentimiento positivo, negativo o neutral. El sentimiento puede ser general o sobre un tema específico, por ejemplo, una persona, un producto o un evento [8].

En este contexto, se presenta esta “Herramienta integral de análisis de sentimientos en redes sociales”. La cual es una aplicación web que interactúa con twitter y está dirigida a una amplia gama de usuarios, ya que puede ser utilizada tanto como una actividad recreativa y de disfrute, como así también para analizar la respuesta en las redes sociales a un producto o servicio de determinada empresa o entidad.

Esta aplicación permite recopilar tweets con diferentes criterios de búsqueda: palabra clave (hashtag), comentarios de un tweet dado o tweets de un usuario dado. Luego nos brinda la posibilidad de analizar los sentimientos de dicho conjunto de opiniones mediante diversos algoritmos:

-Machine learning: la aplicación cuenta con un algoritmo el cual ya fue entrenado con una extensa base de datos de tweets clasificados en positivos, negativos o neutrales.

-Estructuras gramaticales: La tecnología subyacente de este algoritmo se basa en un nuevo tipo de red neuronal recursiva que se basa en estructuras gramaticales. Es el único modelo que puede capturar con precisión los efectos de negación y su alcance para frases positivas y negativas [9].

Por último, esta aplicación nos presenta un marco gráfico e interactivo. Donde mediante diferentes vistas el usuario puede navegar entre los datos y hacer un análisis minucioso de la información obtenida. Esto se implementó con una herramienta de Inteligencia de Negocios (Business Intelligence - BI).

Capítulo 1

Introducción

Las redes sociales se han vuelto masivas debido a una tendencia de desarrollo rápido de sitios web sociales. Facebook, Twitter, LinkedIn y sitios web similares alientan a sus usuarios a crear una lista de “amigos”, “seguidores” y “contactos” que pueden crecer a través de las conexiones indirectas de otros usuarios. Estos sitios pueden proporcionar ayuda, apoyo, oportunidades e incluso una sensación de bienestar que de otra manera no sería posible [1].

Tal ha sido la aceptación por parte de la población mundial a las redes sociales que de las 7,3 mil millones de personas en el planeta, 2,3 mil millones son miembros activos en medios de comunicación social. A su vez este subgrupo está en actual crecimiento, las cifras indican que el mismo es de un 10% anual [2].

Existen estudios empíricos [3, 4] que exploran los factores que afectan a los usuarios que se unen a las redes sociales utilizando cuestionarios en línea. Los resultados muestran que el factor más influyente en el uso continuo de las redes por parte de las personas es el disfrute.

Por otra parte, hay otro tipo de investigaciones que utilizan el disfrute de los usuarios en las redes sociales para obtener información útil en diversos aspectos. Por ejemplo, se realizó una investigación que estudia por medio de las redes sociales la lealtad de los usuarios a determinada marca (empresa, producto, etc) [5]. La investigación proporcionó evidencia contundente de que la mayoría de los usuarios siguen las páginas de sus marcas favoritas a través de las redes. El estudio también mide los comportamientos de lealtad conductual y actitudinal de los usuarios. También se establece su nivel de confianza en la información que obtuvieron sobre las marcas a través de las redes sociales. Las

hipótesis probadas muestran que las marcas y la satisfacción del cliente se relacionan positivamente con la lealtad conductual de los usuarios [5].

La revolución de la información ha dado origen a nuevas economías estructuradas en torno a los flujos de datos, información y conocimiento. Paralelamente, las redes sociales se han fortalecido como formas de organización de la actividad humana [6].

Este capítulo está conformado por tres secciones. En la primera se introduce la motivación que dio origen a este trabajo. En la segunda se detallan los objetivos a cumplir a lo largo de este proyecto. Por último se realiza un resumen de la organización de este trabajo para orientar al lector.

1.1 Motivación

El análisis de redes sociales se aplica cada vez más en las ciencias sociales y se ha aplicado en áreas tan diversas como la psicología, la salud, la organización empresarial y las comunicaciones electrónicas. A pesar de esta realidad, hasta la fecha todavía estamos lejos de poder interpretar su poder público y organizacional, de manera que se pueda aprovechar todo su potencial [6]. La difusión de información, aplicada a redes sociales, estudia cómo las innovaciones, ideas, productos u opiniones, se difunden a través de la sociedad. El ritmo y la especificidad de la difusión, pueden entenderse analizando el patrón de comunicación que existe entre los individuos en una red social [10].

La cantidad de tiempo que los usuarios de todo el mundo invierten diariamente en las redes sociales es cada vez más grande. Tal como se ve en la Figura 1.1 [12] en el año 2012 en promedio los usuarios pasaban 90 minutos en las redes sociales mientras que en 2017 esta cifra aumentó a 135 minutos.

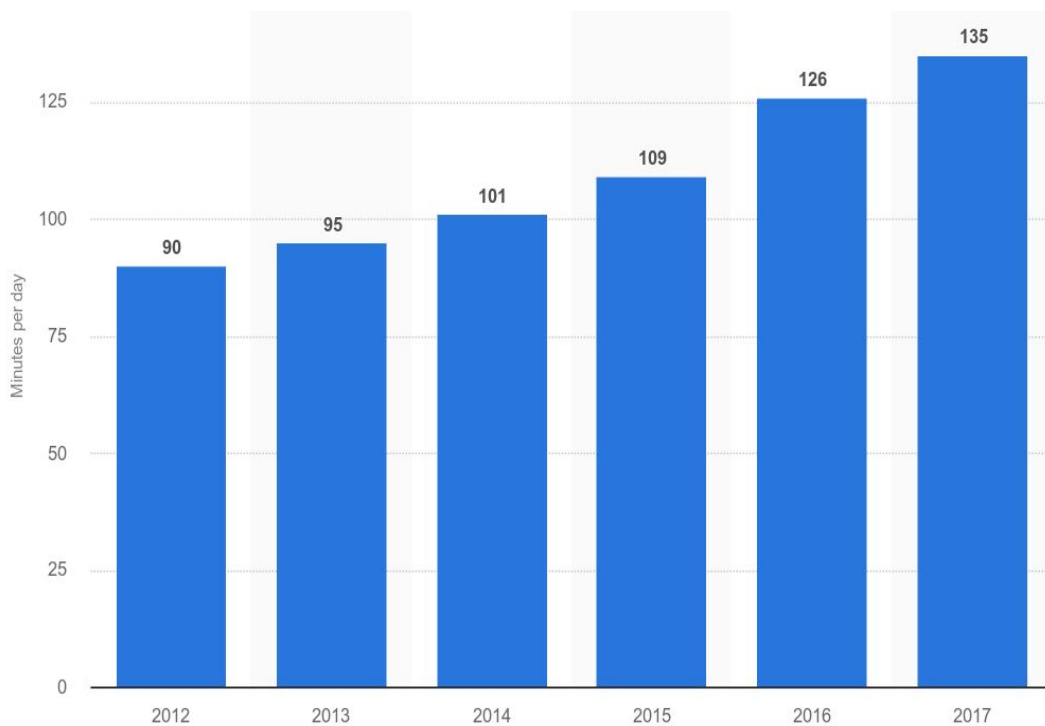


Figura 1.1: Crecimiento en consumo de redes sociales.

El análisis de sentimientos, también llamado minería de opinión, es el campo de estudio que analiza las opiniones, los sentimientos, las evaluaciones, las actitudes y las emociones de las personas a partir del lenguaje escrito [7]. Es una de las áreas de investigación más activas en el procesamiento del lenguaje natural. También está ampliamente estudiada en minería de datos, minería de Internet y minería de texto. La creciente importancia del análisis de sentimientos coincide con el crecimiento de medios de comunicación social, como las reseñas, los debates en foros, los blogs, los microblogs y las redes sociales. Por primera vez en la historia de la humanidad, ahora tenemos un gran volumen de datos de opiniones registrados en forma digital para su análisis.

Para llevar a cabo esta tarea se pre procesan opiniones, escritas en lenguaje natural, mediante diversos algoritmos. Dichas opiniones se clasifican en función de su contenido emocional como: positivos, neutros o negativos. Diferentes autores han estudiado numerosos algoritmos de clasificación los cuales aplican diferentes técnicas o incluso mejoras a técnicas ya conocidas para resolver este problema [8].

Los sistemas de análisis de sentimiento se están aplicando en casi todos los ámbitos comerciales y sociales porque las opiniones son fundamentales para casi todas las actividades humanas y son factores clave que influyen en su comportamientos. Las creencias, percepciones de la realidad, y las decisiones que toman las personas, están en gran medida condicionadas a cómo otros ven y evalúan el mundo. Por esta razón, cuando se debe tomar una decisión, a menudo se buscan las opiniones de los demás. Esto es cierto no solo para individuos sino también para organizaciones [7].

1.2 Objetivos de la Tesis

El objetivo específico radica en crear una herramienta la cual estudie/analice el proceso de difusión de los sentimientos de los usuarios en la red social Twitter, ya sea un sentimiento en sí o esté relacionado a un tema específico. La solución propuesta ayudará a una amplia variedad de usuarios a realizar un análisis minucioso del contenido de la red social twitter. Los mismos pueden ser simplemente personas que por entretenimiento o curiosidad usen esta herramienta, como así también pueden ser empresas o corporaciones que la utilicen para sacar ventajas en sus negocios. De lo contrario, sin contar con la ayuda de esta herramienta, el usuario debería leer cada uno de los tweets (sin mencionar que debería llevar registro de lo leído, como así también el género del usuario, país en que vive, edad, etc.).

Dicha tarea comienza con la obtención de una base de datos lo suficientemente grande, conformada por opiniones (en lenguaje natural) de usuarios de Twitter. Estas opiniones deben estar relacionadas mediante un tema en común, para luego poder analizar el sentimiento de los usuarios hacia el mismo. Por el contrario los usuarios pueden estar relacionados o no, y deben tener atributos que los diferencien unos de los otros (edad, género, país, etc), permitiendo de esta manera que los resultados sean un reflejo fiel de la sociedad.

Luego, con el fin de conocer el carácter de las opiniones de los usuarios respecto a un determinado tema, se debe proceder a la investigación e implementación de diversos algoritmos para clasificar dichas opiniones en positivas, neutras o negativas.

Según [11] los problemas principales de este tipo de algoritmos son la precisión de clasificación, la escasez de datos y el sarcasmo, ya que clasifican incorrectamente

la mayoría de las opiniones. Los resultados de su investigación mostraron un porcentaje muy elevado de opiniones clasificados incorrectamente como neutros.

Uno de los objetivos de esta tesis es combinar algoritmos y estudiar en qué circunstancias funcionaba mejor cada uno de ellos, intentando mejorar la precisión de la clasificación de la entrada.

Como objetivo principal, se desarrollará una herramienta web dirigida a un usuario específico que le facilite la tarea de analizar los resultados obtenidos de forma gráfica. Este sistema permitirá combinar datos operacionales con herramientas analíticas para presentar información compleja y competitiva para los planificadores y mejorar la calidad de los aportes al proceso de toma de decisiones. El mismo también incluye diferentes gráficos, encargados de plasmar en un contexto geográfico las opiniones clasificadas.

Se realizó un análisis de las utilidades y del flujo que debería tener esta aplicación y surgieron dos módulos principales, la obtención de información de la red social y el análisis de dicha información. Estos dos casos fueron la base de los requerimientos para desarrollar la aplicación adecuada.

Capítulo 2

Marco Teórico

En este capítulo se detallarán los conceptos y fundamentos teóricos en los cuales se basa este trabajo final. También se establecerá el contexto para el cual fue creada la aplicación resultante, como así también al tipo de usuarios a los que está dirigida. Por último, se indicarán diferencias con sistemas de similares características. Al finalizar su lectura se deberá lograr una idea general del contexto del proyecto a trabajar.

2.1 Análisis de redes sociales

“Red social” es un término general que abarca muchos elementos y tiene diversas funciones, ya que cada nodo tiene un valor relativo distinto y puede ser utilizado para representar eventos, ideas, objetos o personas. Las redes sociales son comunidades que residen en la web, conectadas a través de correo electrónico, sitios web y aplicaciones de red como Twitter, FaceBook o LinkedIn.

La característica definitoria del análisis de redes sociales es su enfoque en la estructura de las relaciones, que van desde el conocimiento casual hasta vínculos estrechos. El análisis de redes sociales asume que las relaciones son importantes. Intenta comprender qué factores facilitan o impiden los flujos de conocimiento que vinculan a las unidades interactivas, es decir, quién conoce a quién y quién comparte qué información y conocimiento con qué medios de comunicación (por ejemplo, datos e información, mensajes de voz , o video comunicaciones) [6].

Los resultados de un análisis de redes sociales podrían utilizarse, por ejemplo, para identificar las personas, los equipos y las unidades que desempeñan papeles

centrales en cierto contexto, para mejorar la innovación y el aprendizaje o para refinar estrategias.

2.2 Análisis de Sentimientos

Los campos de análisis de sentimiento y minería de opinión están en auge. Hay una avalancha virtual de datos disponibles de una amplia variedad de fuentes en la web que se puede usar para analizar las actitudes detrás del material textual. Millones de publicaciones de Twitter, millones de publicaciones en Facebook y miles de millones de páginas web y otros documentos pueden ser revisados para determinar las opiniones detrás de las palabras. Por ejemplo numerosos sitios como Yelp, Tripadvisor y Foursquare almacenan millones de *reviews* (opiniones sobre productos y servicios). Las mismas son el contenido más influyente con el que se encuentran los consumidores en el momento que visitan el sitio web de algún producto o servicio.

Este análisis puede ser extremadamente útil para investigadores y practicantes. Los profesionales en marketing pueden monitorear las comunicaciones textuales y determinar las actitudes actuales hacia sus productos. Los políticos pueden analizar las comunicaciones textuales para determinar la popularidad y sentimientos hacia su candidatura. Los investigadores también pueden estudiar datos textuales para encontrar diferencias, patrones o tendencias en un amplia variedad de textos, desde textos políticos hasta presentaciones, desde documentos hasta sitios web [13].

Hay muchas fuentes y algoritmos para analizar los sentimientos. Pero todo análisis de sentimiento incluye cierta incertidumbre ya que el análisis de sentimiento absoluto no es posible. Tal como se demuestra en [33] incluso para dos personas la misma oración puede tener diferentes interpretaciones. Según dicha investigación se han realizado varios estudios dirigidos por diversas personas y compañías, y se llegó a la conclusión de que la tasa de concordancia humana está entre el 70% y el 79%.

Hay diferentes razones que inducen a esta limitación, incluyendo negación, sarcasmo, combinaciones de palabras y subjetividad relativa. Como resultado, es útil usar múltiples estrategias de análisis para confirmar cualquier sentimiento hallado [13].

El análisis del sentimiento, que también se conoce como minería de opinión, implica la construcción de un sistema para recopilar y examinar opiniones sobre un tema específico realizadas en publicaciones de blogs, comentarios, reseñas o redes sociales [19].

2.3 Algoritmos de clasificación utilizados

En la investigación realizada por Vinodhini y Chandrasekar [19], se han estudiado varias técnicas de machine learning para clasificar texto y las mismas han logrado un gran éxito en esta tarea. Los algoritmos que utilizan métodos de machine learning (aprendizaje automático) aplicadas al análisis de sentimiento consisten en la utilización de dos técnicas complementarias:

- **Aprendizaje supervisado:** requiere entrenamiento previo para clasificar los datos.
- **Técnicas de NLP:** técnicas de procesamiento del lenguaje natural.

A continuación se presentarán dos bibliotecas que se investigaron para este proyecto, detallando su funcionamiento y el método que utilizan para realizar la clasificación. Ambos utilizan técnicas de machine learning para dicha tarea.

2.3.1 Stanford CoreNLP

Stanford CoreNLP es una biblioteca de procesamiento de lenguaje natural (NLP - Natural Language Processing) para Java. Proporciona un conjunto de herramientas que permiten detectar las formas básicas de palabras y sus partes del discurso, ya sean nombres de compañías, personas, etc., normalizar fechas, horas y cantidades numéricas, marcar la estructura de oraciones en términos de frases y dependencias sintácticas, indicar qué frases nominales se refieren a las mismas entidades, indicar sentimiento, extraer relaciones particulares o de clase abierta entre menciones de entidades, obtener las citas que las personas dicen, etc. [23].

El algoritmo de clasificación de sentimientos de Stanford clasifica un texto de entrada escrito en el idioma inglés en cinco diferentes categorías. También devuelve la confianza con la que realizó esta predicción. Dicha confianza es en realidad la probabilidad estimada de cada categoría, por lo tanto la suma de las mismas es igual a uno. A continuación un ejemplo de su funcionamiento:

Tweet: *“Enjoy a lot with my family”*

- Muy negativo -> 0,001
- Negativo -----> 0,004
- Neutro-----> 0,222
- **Positivo-----> 0,731**
- Muy Positivo---> 0,041

El algoritmo de Stanford clasificó el texto en la categoría de positivos con una confianza del 73%. La segunda categoría con mayor confianza es la de neutros con un 22% y las restantes tienen menos del 10% de confianza.

La mayoría de los sistemas de predicción del sentimiento funcionan simplemente mirando las palabras en forma aislada, dando puntos positivos para las palabras positivas y puntos negativos para las palabras negativas y luego resumiendo estos puntos. De esta forma, se ignora el orden de las palabras y se pierde información importante. En contraste, el modelo de aprendizaje profundo de *Stanford coreNLP* construye una representación de oraciones basadas en la estructura de la oración. Calcula el sentimiento en función de cómo las palabras componen el significado de las frases más largas. De esta manera, el modelo no es tan fácil de engañar como los modelos anteriores. Por ejemplo, este modelo aprendió que gracioso e ingenioso son palabras positivas, pero la siguiente oración sigue siendo negativa en general:

This movie was actually neither that funny, nor super witty.

Esta película en realidad no era tan graciosa, ni super ingeniosa.

Avanzar hacia la comprensión de la composicionalidad en tareas tales como la detección del sentimiento requiere un entrenamiento supervisado más rico y recursos de evaluación y modelos de composición más potentes. Para ello, Stanford CoreNLP utiliza el *Sentiment Treebank*¹ que es una base de datos de árboles de sentimiento. El cual está conformado por reseñas de películas e incluye etiquetas de sentimiento para

¹ <https://nlp.stanford.edu/sentiment/treebank.html>

215,154 frases en los árboles de análisis de 11,855 oraciones y presenta nuevos desafíos en la investigación de la composicionalidad del sentimiento.

Para capturar los efectos de composición con mayor precisión, utiliza un modelo llamado *Recursive Neural Tensor Network* (RNTN). Este modelo toma frases de entrada de cualquier longitud. Representa una frase a través de vectores de palabras y luego calcula nuevos vectores a partir de los iniciales, formando un árbol de análisis sintáctico. De esta manera RNTN puede predecir con mayor exactitud los efectos semánticos de las composiciones presentes en el *Sentiment Treebank* con el cuál fue entrenado [24].

Un ejemplo de Stanford CoreNLP prediciendo el sentimiento de una oración con el modelo RNTN puede verse en la Figura 2.1.

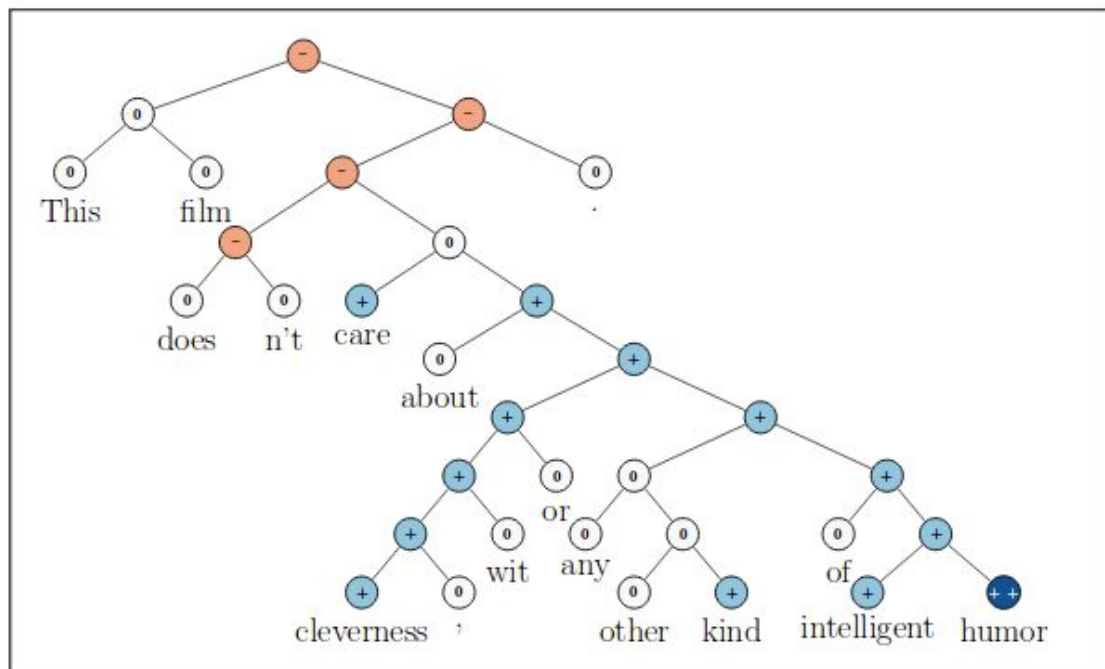


Figura 2.1: Ejemplo de árbol sintáctico.

En la Figura 2.1 se ve un ejemplo de cómo la RNTN predice entre 5 clases de sentimientos en cada nodo de un árbol de análisis sintáctico, capturando la negación y su alcance en esta oración.

Este modelo supera a todos los métodos anteriores en varias métricas. Se evaluó con una base de datos puntual y demostró una mejora en la técnica de clasificación en positiva / negativa de una sola oración desde el 80% hasta el 85,4%. La precisión de predicción de las etiquetas para todas las frases alcanzó el 80.7%. Por último, es el

único modelo que puede capturar con precisión el efecto de las conjunciones contrastivas así como de la negación y su alcance en varios niveles de árbol para frases positivas y negativas [24].

2.3.2 LingPipe

LingPipe es un conjunto de herramientas para procesar texto utilizando la lingüística computacional. Su arquitectura está diseñada para ser eficiente, escalable, reutilizable y robusta.

La biblioteca disponible de dichas herramientas incluye diversos elementos, algunos de ellos son:

- API Java con código fuente y pruebas unitarias
- Modelos multilingües, multidominio y multi-género
- Entrenamiento con nuevos datos para nuevas tareas
- Mejor resultado con estimaciones estadísticas de confianza

Una de las tareas principales para la cual se utilizan las herramientas de LingPipe es para clasificar texto automáticamente en diferentes categorías, utilizando clasificadores de modelos de lenguaje [22]. Un modelo de lenguaje es un mecanismo para definir la estructura del lenguaje, es decir, para restringir adecuadamente las secuencias de unidades lingüísticas más probables.

El proceso de clasificación se divide en dos etapas:

- **Entrenamiento:** generación de un clasificador. El mismo está entrenado para clasificar texto en n categorías. Cada categoría posee su propio modelo de lenguaje.
- **Clasificación:** se utiliza el clasificador generado para predecir a qué categoría pertenece un texto de entrada dado.

Para entrenar el algoritmo, la biblioteca requiere como información de entrada un set de entrenamiento. El mismo está conformado por un conjunto de textos clasificados, normalmente generados por un ser humano.

Una vez entrenado el clasificador, la biblioteca utiliza un método llamado clasificación condicional [16]. El mismo consta en una clasificación calificada que estima probabilidades condicionales de categorías dadas una entrada. Por defecto, las probabilidades condicionales son los puntajes que se le asignan a cada categoría. Dado un texto de entrada, LingPipe devuelve como salida la categoría con el puntaje más alto. Además, devuelve la confianza con la que realizó esta predicción. La confianza es precisamente la probabilidad condicional asociada a cada categoría y la suma de las mismas es igual a uno. Este proceso se muestra en la Figura 2.2.

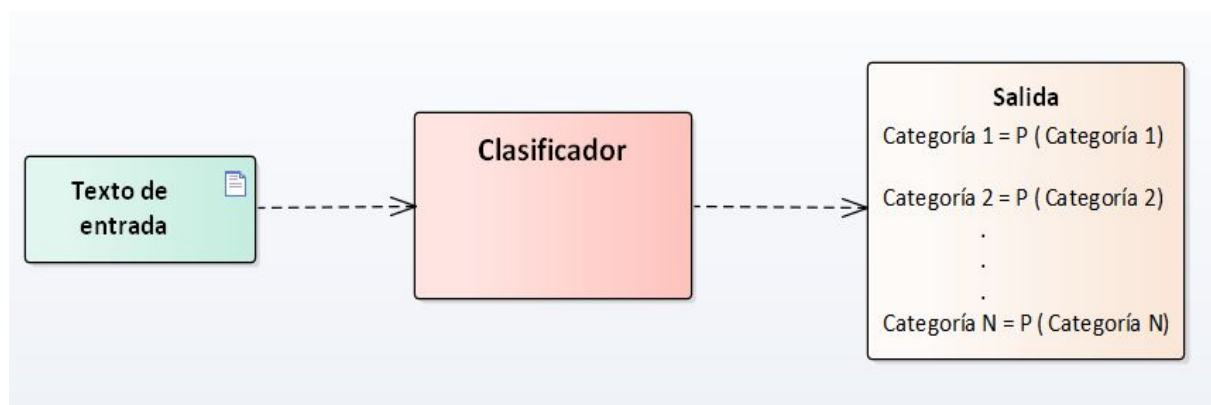


Figura 2.2:Flujo de clasificación.

2.4 REST – Servicios Web

El uso de REST (Representational State Transfer - Transferencia de Estado Representacional) creció exponencialmente dentro de la ingeniería de software a partir del 2000. Este nuevo enfoque de desarrollo de proyectos y servicios web fue definido por Roy Fielding, en su disertación ‘Architectural Styles and the Design of Network-based Software Architectures’. En el campo de las APIs, REST es, a día de hoy, la alternativa más utilizada para el desarrollo de servicios de aplicaciones [25].

Fielding generalizó los principios arquitectónicos de la Web y los presentó como un marco de restricciones, o un estilo arquitectónico. A través de este marco, describió cómo se construyen y operan los sistemas de información distribuida como la Web. Describió la interacción entre los recursos y el papel de los identificadores únicos en dichos sistemas. También habló sobre el uso de un conjunto limitado de

operaciones con semántica uniforme para construir una infraestructura ubicua que pueda soportar cualquier tipo de aplicación. Fielding se refirió a este estilo arquitectónico como REST, el cual describe la Web como una aplicación de hipermedia distribuida cuyos recursos vinculados se comunican intercambiando representaciones de estado de recursos [34].

REST es una alternativa en auge a otros protocolos estándar de intercambio de datos, por ejemplo RPC, SOAP y Messaging System, que disponen de una gran capacidad pero también mucha complejidad. A veces es preferible una solución más sencilla de manipulación de datos como REST.

Algunas de sus características son:

- **Protocolo cliente/servidor sin estado:** cada petición HTTP contiene toda la información necesaria para ejecutarla, lo que permite que ni cliente ni servidor necesiten recordar ningún estado previo para satisfacerla.
- Las operaciones más importantes relacionadas con los datos en cualquier sistema REST y la especificación HTTP son cuatro: **POST** (crear), **GET** (leer y consultar), **PUT** (editar) y **DELETE** (eliminar).
- **Los objetos en REST siempre se manipulan a partir de la URI.** Es la URI y ningún otro elemento el identificador único de cada recurso de ese sistema REST. La URI nos facilita acceder a la información para su modificación o borrado, o, por ejemplo, para compartir su ubicación exacta con terceros.
- **Interfaz uniforme:** para la transferencia de datos en un sistema REST, este aplica acciones concretas (POST, GET, PUT y DELETE) sobre los recursos, siempre y cuando estén identificados con una URI. Esto facilita la existencia de una interfaz uniforme que sistematiza el proceso con la información.

Las ventajas que ofrece REST para el desarrollo son las siguientes:

- **Separación entre el cliente y el servidor:** el protocolo REST separa totalmente la interfaz de usuario del servidor y el almacenamiento de

datos. Eso tiene algunas ventajas cuando se hacen desarrollos. Por ejemplo, mejora la portabilidad de la interfaz a otro tipo de plataformas, aumenta la escalabilidad de los proyectos y permite que los distintos componentes de los desarrollos se puedan evolucionar de forma independiente.

- **Visibilidad, fiabilidad y escalabilidad.** La separación entre cliente y servidor tiene una ventaja evidente y es que cualquier equipo de desarrollo puede escalar el producto sin excesivos problemas. Se puede migrar a otros servidores o realizar todo tipo de cambios en la base de datos, siempre y cuando los datos de cada una de las peticiones se envíen de forma correcta. Esta separación facilita tener en servidores distintos el *front* y el *back* y eso convierte a las aplicaciones en productos más flexibles a la hora de trabajar.
- **La API REST es independiente del tipo de plataformas o lenguajes:** se adapta al tipo de sintaxis o plataformas con las que se estén trabajando, lo que ofrece una gran libertad a la hora de cambiar o probar nuevos entornos dentro del desarrollo. Con una API REST se pueden tener servidores PHP, Java, Python o Node.js. Lo único que es indispensable es que las respuestas a las peticiones se hagan siempre en el lenguaje de intercambio de información usado, normalmente XML o JSON.

2.5 WebSocket

El protocolo WebSocket plantea un modelo elegante y sencillo de comunicaciones para la Web que no rompe con las tecnologías ya existentes. Permite que dos aplicaciones establezcan una comunicación bidireccional independientemente de la plataforma en la que estén ejecutándose y del lenguaje en el que hayan sido escritas. Además, existen múltiples implementaciones para prácticamente cualquier lenguaje que permiten a los desarrolladores centrarse en sus aplicaciones olvidándose de implementar las comunicaciones.

Esto abre un abanico de posibilidades para la integración de aplicaciones, permitiendo a estas intercambiar información en tiempo real y de forma sencilla, contribuyendo además a la estandarización de los mecanismos de comunicación [26].

Ventajas y desventajas

Tal como se mencionó anteriormente, el protocolo de websockets permite establecer comunicaciones bidireccionales en tiempo real en la Web, posibilidad que antes solo existía de forma simulada y bastante costosa. Optar por este protocolo permite reducir la saturación de cabeceras que ocurriría si se utilizase HTTP en su lugar, especialmente para aplicaciones que requieren un gran volumen de comunicaciones. Este protocolo se estandarizó para evitar que cada aplicación utilice una solución de integración diferente, con los problemas de compatibilidad que ello conlleva. Por otra parte, su funcionamiento es extremadamente sencillo: se establece una conexión, se envían/reciben mensajes y se cierra la conexión. Por último, al funcionar bajo los mismos puertos que HTTP evita problemas relacionados con *firewalls*, facilitando así el funcionamiento de productos basados en arquitecturas orientadas a servicios (SOA), entre otros.

Analizando las desventajas, es necesario gestionar y mantener un gran número de conexiones que han de permanecer abiertas mientras ambas partes sigan interactuando. Esto puede llegar a ser un problema en determinados casos, teniendo en cuenta que el número máximo de conexiones simultáneas que admite un puerto TCP es de 64.000 y que, además, mantener las conexiones abiertas requiere memoria del servidor.

2.6 Alternativas para el sistema

En este caso se buscaron aplicaciones que realizan algún tipo de análisis de sentimientos en la red social de twitter. Las herramientas similares existentes se clasifican en dos grupos:

Herramientas pagas: Como se mencionó anteriormente, el análisis de redes sociales es un complemento vital para casi cualquier tipo de empresa. Por lo tanto no es sorprendente que exista una gran variedad de herramientas pagas las cuales apuntan a prestar servicios a grandes corporaciones.

Se vió que este tipo de herramientas ofrecen versiones gratuitas, que permiten probar los clasificadores de sentimientos, haciendo pequeñas estimaciones de porciones de textos. Posteriormente cobran sus servicios. Por ejemplo la herramienta *Revealed Context*² ofrece los servicios que se muestran en la Figura 2.3 con sus tarifas asociadas:

Sign up here and we'll send you your API key to get started.
No long-term contracts — Accurately classify any social media conversation according to its sentiment, emotion, and intensity.

		Standard Classifiers [?]		Standard & Relevance [?] Classifiers	
	MAXIMUM DOCUMENTS PER DAY	COST PER MONTH		COST PER MONTH	
TRIAL	250	Free	SELECT	\$100	SELECT
BRONZE	1,000	\$100	SELECT	\$300	SELECT
SILVER	50,000	\$800	SELECT	\$2,000	SELECT
GOLD	150,000	\$2,250	SELECT	\$6,000	SELECT
PLATINUM	500,000	\$8,000	SELECT	\$20,000	SELECT
ENTERPRISE	More Than 500,000	Contact Us	SELECT	Contact Us	SELECT

Figura 2.3: Costos herramienta paga alternativa.

Como se ve, existen diferentes categorías, cada una permite utilizar la herramienta una mayor cantidad de veces al día que la categoría inferior. Notar que a su vez la tarifa que se debe abonar también es mayor, incluso para acceder a *Enterprise* hay que contactarse con el proveedor para que decida cuál será el costo para la empresa que está interesada en el servicio.

Herramientas gratuitas: Por otra parte se encontraron diversas herramientas gratuitas en la web que aplican análisis de sentimientos en diferentes elementos de twitter. Algunas de ellas son:

- **Sentiment140**³: dada una palabra clave selecciona una muestra pequeña de tweets (menor a 100) y muestra el sentimiento asociado. Permite seleccionar entre idioma español o inglés. Muestra cada uno de los tweets utilizados y los

² <https://www.revealedcontext.com/sign-up>

³ <http://www.sentiment140.com/>

colorea según el sentimiento.

- **25Trends⁴**: muestra el sentimiento asociado a una palabra clave. Además devuelve ciertas métricas asociadas a los tweets con dicha palabra. Por ejemplo los usuarios más populares que la mencionaron, la tendencia de temas vinculados a la palabra, las palabras más usadas, en qué lenguajes están hechos, etc.
- **Tweet-mood⁵**: dice que tan popular es un tweet, permite rankear tweets entre sí en cuanto a su popularidad (máximo tres a la vez). Además permite ingresar un lapso de tiempo y un nombre de usuario y muestra que tan popular fue el usuario ingresado en ese período.
- **Mr. Tuit⁶**: clasifica un texto dado en positivo, negativo o neutro. No está conectado con twitter, ya que el texto debe ser ingresado a mano por el usuario.

Todas las herramientas gratuitas tienen varios aspectos en común:

- Limitan al usuario en cuanto al volumen de datos y al tipo de tweets que desea analizar.
- Los resultados del análisis se muestran de manera estática, no permiten filtrar o hacer subconsultas de los datos de salida lo cual dificulta el análisis de múltiples datos.
- La interfaz de salida es básica y tiene información mínima.

No existe una herramienta integral que provea soporte para analizar respuestas a un tweet dado ni tampoco para analizar los tweets publicados por un mismo usuario. Tampoco existen herramientas cuya salida pueda ser analizada con un marco gráfico robusto, que permita al usuario interactuar con los datos.

⁴ <http://twitter.25trends.me/services/twitter/public/>

⁵ <http://www.tweet-mood-check.appspot.com/tweetscore>

⁶ <http://www.mrtuit.com/>

Capítulo 3

Solución propuesta

En este capítulo se da una visión de alto nivel de la solución planteada para el contexto mencionado anteriormente. En principio, se detallan los requerimientos principales que guiaron el desarrollo de esta herramienta. Luego se hace mención de las decisiones tomadas para lograr la obtención de información, el análisis de la misma y la forma en que se visualizan los resultados. Finalmente se plantea la aplicación propuesta y sus características.

3.1 Relevamiento de requerimientos

En la primera etapa del proyecto se comenzó investigando aplicaciones y herramientas similares a la propuesta en este proyecto para determinar las características iniciales que tendría la misma. El objetivo fue tener como mínimo la misma funcionalidad que tienen las herramientas existentes. Por otra parte se encontraron algunos puntos a mejorar, con el fin de crear una aplicación con características que no fueron encontradas en ninguna otra similar.

A continuación se detallarán los requerimientos principales, dejando en claro qué tipo de aplicación se creó y con qué funcionalidad:

- Permitir al usuario recopilar tres tipos diferentes de tweets: comentarios de un tweet dado, tweets que contengan determinado hashtag o el historial de tweets de un usuario en particular.
- Limpiar los tweets obtenidos para dejar oraciones libres de emoticones, menciones (@nombre_usuario_mencionado), direcciones url, etc.

- Parametrizar la cantidad de tweets a descargar, para que el usuario pueda obtener el volumen de datos deseado.
- Clasificar los tweets mediante algoritmos y/o heurísticas en positivos, neutros o negativos.
- Equipar la herramienta con una interfaz gráfica robusta para visualizar y analizar los resultados obtenidos.
- La aplicación debe ser de fácil acceso para el usuario, sin la necesidad de descargar e instalar numerosos paquetes para su funcionamiento.

Los requerimientos mencionados en la sección anterior pueden resumirse a grandes rasgos en los siguientes tres requerimientos:

- **Obtención de información:** la tarea se resume a obtener y almacenar un conjunto de tweets, teniendo en cuenta que los mismos pueden ser reunidos por diferentes temáticas y de diferentes usuarios. El usuario debe poder obtener cualquier volumen de datos, siempre y cuando las restricciones de la API lo permitan.
- **Análisis de la información:** consiste en realizar un procesamiento de la información obtenida para poder calcular métricas y estadísticas que permitan sacar conclusiones.
- **Visualización de los resultados:** plasmar de la mejor manera posible los resultados obtenidos para mostrarlos al usuario final.

En las secciones próximas se desarrollará como se pretende cumplir con dichos requerimientos y los motivos de las decisiones tomadas.

3.1.1 Obtención de Información

Para esta aplicación se propuso utilizar una red social como principal fuente de información. Las características principales de las redes sociales más populares se pueden ver en la Tabla 3.1.

Red Social	Provee API	Texto en publicaciones	Acceso a información de usuario
	Si	Si, bastante	No, se necesita consentimiento explícito de cada usuario
	Si	Muy poco	Si
	Si	Mayormente texto	Si

Tabla 3.1: Tabla comparativa redes sociales.

La red social elegida fue Twitter, ya que las publicaciones contienen mayormente texto y permite acceder libremente a la gran mayoría de la información publicada por los usuarios. El límite lo pone el usuario decidiendo qué tanta información comparte en sus tweets (mensajes-publicaciones).

Twitter es un canal de microblogging para compartir actualizaciones de estado individuales con el mundo. El mismo, se ha convertido en un nuevo centro de atención a través del cual se conocen acontecimientos recientes. Los usuarios de Twitter siguen a otros y son seguidos. A diferencia de la mayoría de los sitios de redes sociales en línea, como Facebook o MySpace, la relación de seguimiento no requiere reciprocidad. Ser un seguidor en Twitter significa que el usuario recibe todos los tweets de aquellos a los que el usuario sigue.

La simple acción de responder a un tweet provee diferentes opciones de marcado:

- **RT significa retweet:** permite a los usuarios difundir información de su elección más allá del alcance de los seguidores del tweet original.
- '@' seguido del identificador de un usuario, menciona al usuario.
- '#' seguido de una palabra representa un hashtag.

Este vocabulario de marcado combinado con un límite estricto de 140 caracteres por publicación, limita a los usuarios a expresarse con brevedad.

La popularidad de Twitter y el gran volumen de datos que maneja llama la atención y curiosidad de un gran número de estudiantes e investigadores. ¿Cómo se conectan las personas en Twitter? ¿Quiénes son las personas más influyentes? ¿De qué habla la gente? ¿Cómo se difunde la información a través de los retweets? [14]. En este proyecto se propone una herramienta de análisis de sentimientos robusta, que ayudará a comprender un poco mejor que sienten los usuarios en sus publicaciones. La misma, permite al usuario analizar tweets de tres tipos posibles:

- **Tweets de un usuario:** dado el nombre de un determinado usuario, obtiene los tweets publicados por el mismo.
- **Respuestas a un tweet:** obtiene las respuestas de un tweet dado.
- **Hashtag:** obtiene tweets que contienen un hashtag dado.

3.1.2 Análisis de la información

Una vez determinada la fuente de información, se propuso aplicar técnicas de análisis de sentimientos sobre la misma. Para esta tarea se evaluaron diferentes métodos y herramientas con el fin de seleccionar la/las que mejor se adapten a los objetivos de este proyecto.

- **Herramientas externas:** En primer lugar se estudió la posibilidad de utilizar alguna herramienta externa, que provea algún tipo de interfaz para integrar con esta aplicación. Tal como se vio en el capítulo Marco Teórico, la gran mayoría de las aplicaciones que predicen sentimientos con buena precisión son pagas. Motivo por el cual se descartó esta opción.
- **Bibliotecas:** Luego se realizó una búsqueda de bibliotecas con algoritmos de clasificación. Las mismas debían tomar como entrada un texto y clasificarlo según el sentimiento en positivo, neutro o negativo. Las mismas son simples de instalar e integrar ya que no requieren una etapa de entrenamiento para su uso.
- **Algoritmos para entrenar:** Por último se investigó la existencia de algoritmos públicos que puedan ser entrenados previamente a su ejecución. Los mismos permiten entrenar un clasificador de sentimientos con diferentes bases de datos,

lo cual conlleva a la obtención de diferentes algoritmos con diferentes precisiones.

Se decidió implementar dos soluciones automatizadas para esta tarea. La primera consiste en una biblioteca de clasificación de sentimientos y la segunda en un algoritmo a entrenar. Con estas dos opciones se hace posible experimentar y hacer comparaciones en búsqueda de resultados más precisos.

3.1.3 Visualización de los resultados

En este proyecto final de tesis surgió la idea de dar mayor valor agregado a la visualización de los resultados, la integración de una herramienta de Inteligencia de Negocio (Business Intelligence - BI). La misma permite visualizar los datos de salida de manera dinámica, ya que brinda la posibilidad de navegar en ellos por medio de los diferentes gráficos, filtrarlos a elección del usuario y comparar diferentes consultas realizadas. Estas cualidades no se encontraron en ninguna de las aplicaciones gratuitas estudiadas, lo cual motivó este proyecto.

Este tipo de herramientas permiten que los gerentes y analistas de negocios monitoreen datos y estadísticas, tomen decisiones, analicen las tendencias del mercado, estudien la inserción de nuevos productos y mantengan relaciones rentables y duraderas con los clientes para mantenerse competitivos en un mundo global [17].

La herramienta de BI utilizada será introducida y desarrollada con más detalle en la sección 4.1.3: “Herramienta de BI”.

3.2 Aplicación propuesta

Una vez que se definió la funcionalidad de la aplicación y los métodos propuestos para implementarla, se prosiguió por decidir de qué manera accederán los usuarios finales a la misma. Se analizaron las ventajas y desventajas de dos tipos de aplicaciones. La primer opción es la de crear una aplicación de escritorio y la segunda una aplicación web, el análisis comparativo se resume en la Tabla 3.2.

Escritorio	Web
-Requiere descargar e instalar paquetes	+No requiere descargar e instalar paquetes
-No es compatible con múltiples plataformas	+ Es compatible con múltiples plataformas
-No se actualiza de forma automática	+ Se actualiza de forma automática
-Consume memoria del cliente para el procesamiento	+ No consume memoria del cliente para el procesamiento
-No es compatible con todos los Sistemas Operativos	+ Es compatible con todos los Sistemas Operativos
+Tiene buen tiempo de respuesta	- El tiempo de respuesta es mayor que en una aplicación de escritorio
+ Es seguro	+ Es seguro

Tabla 3.2: Tabla comparativa aplicación de escritorio versus web.

Se optó por crear una aplicación Web. Si bien esto implica tener un mayor tiempo de respuesta, en este contexto se considera que no es relevante ya que esta aplicación no funciona mediante consultas constantes. Es decir, luego de realizar una consulta el usuario pasa cierto tiempo haciendo un análisis interactivo de la salida antes de realizar una consulta nuevamente.

Entrando en detalle sobre la opción elegida, una aplicación web es aquella que se puede utilizar accediendo a un servidor web a través de Internet o una Intranet. Su popularidad se debe a la facilidad de uso de los navegadores web como clientes "delegados" y a la capacidad de actualizar y mantener aplicaciones web sin distribuir e instalar software en miles de clientes potenciales.

La arquitectura implementada en todas las páginas web es el modelo Cliente-Servidor. En este modelo se aloja una aplicación distribuida en la que los clientes pueden ser vistos como módulos que realizan peticiones a un programa servidor, a fin de promover el intercambio de información entre ellos. El

procesamiento de datos es manejado por el servidor, y los resultados se devuelven a los clientes, lo cual se hace para acelerar la tasa de rendimiento y permitir a muchos usuarios tener acceso a la misma base de datos al mismo tiempo [18].

En la Figura 3.1 puede verse un esquema general de este modelo. Los clientes son los usuarios finales que a través del navegador web envían peticiones al servidor.

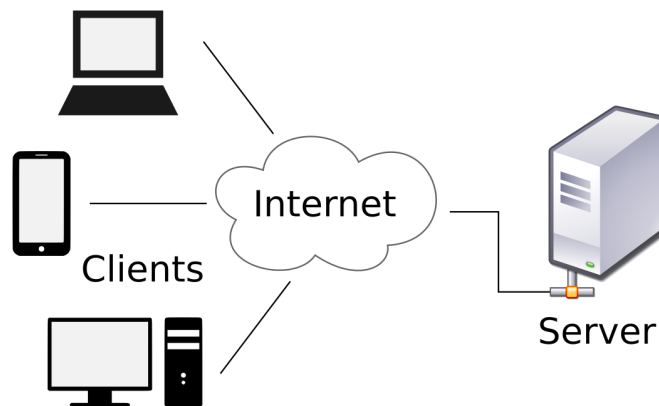


Figura 3.1: Modelo Cliente-Servidor.

Reuniendo todos los componentes de la aplicación propuesta hasta el momento, la arquitectura inicial es la que se observa en la Figura 3.2. Allí se puede ver un servidor compuesto por tres componentes principales: la Api de Twitter, los algoritmos encargados de la clasificación y la herramienta de BI.

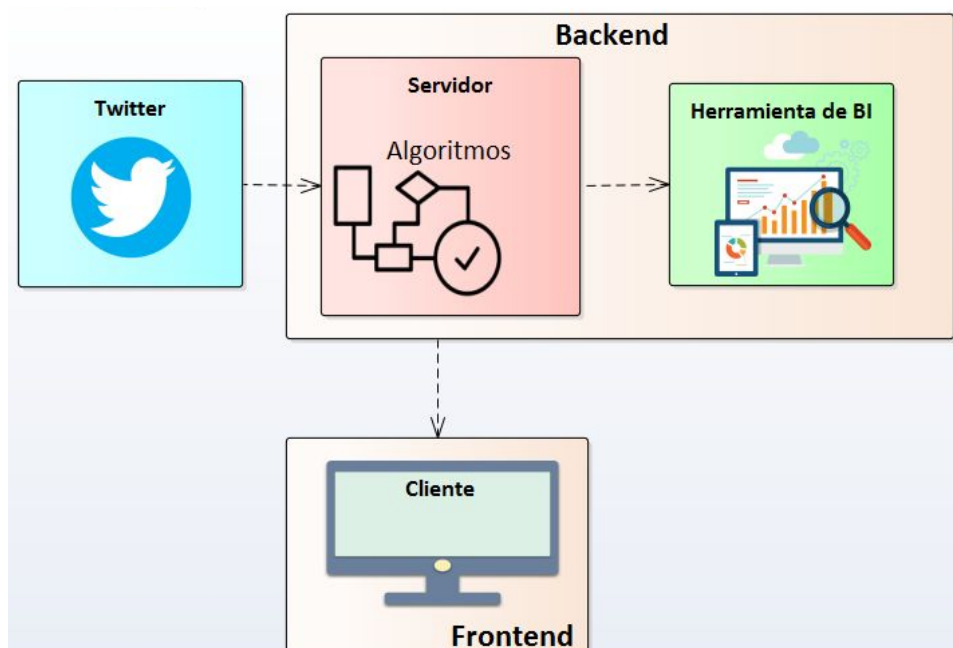


Figura 3.2: Arquitectura inicial del sistema.

Capítulo 4

Diseño e implementación

En este capítulo se describe con detalle el diseño elegido para el desarrollo del sistema y la implementación del mismo, teniendo en cuenta las decisiones que se fueron tomando a lo largo de la creación de esta aplicación.

Se pretende lograr que el lector tenga una visión general de las características técnicas del proyecto y su evolución pudiendo ubicar en el tiempo la dedicación a cada etapa y el desarrollo realizado en cada una de ellas.

4.1 Tecnologías de desarrollo

El ambiente planteado para el desarrollo de este sistema consta de todas las partes que requiere cualquier aplicación Web. Tal como se explicó en el capítulo anterior esta aplicación se establece bajo un modelo Cliente-Servidor.

Para su implementación se vió la necesidad de utilizar diferentes tecnologías y *frameworks* que ayuden a que el desarrollo sea más ágil y brinden mejores prestaciones para el producto final.

Para la implementación de esta aplicación se utilizó Eclipse⁷ que es un entorno de desarrollo compuesto por un conjunto de herramientas de programación de código abierto multiplataforma para desarrollar aplicaciones de cliente enriquecido. Se tomó esta decisión ya que es una herramienta conocida que se utilizó en el transcurso de la carrera.

⁷ Eclipse Foundation, Inc- IBM. <https://www.eclipse.org/> 2004-2017

El manejo de versiones durante el desarrollo del sistema se realizó con GitHub⁸, un sistema de control de versiones distribuidas de código abierto y gratuito diseñado para manejar todo tipo de proyectos.

4.1.1 Cliente

Se decidió utilizar un framework robusto que ayude a implementar el sistema del lado del cliente. Una de las principales ventajas por la cual se tomó esta decisión es que permiten desarrollar funcionalidad y que la misma se aloje del lado del cliente, es decir las acciones no necesariamente requieren una consulta al servidor, lo que beneficia los requerimientos de usabilidad y escalabilidad.

Dentro de la variedad de herramientas que se encuentran en la actualidad para esta tarea se optó por trabajar con *Angular*⁹. El mismo es un framework que permite hacer todo lo necesario en una aplicación cliente compleja, es de código abierto y está orientado al desarrollo ágil de páginas web. Se trabajó específicamente con *Angular CLI* que es la herramienta de línea de comandos estándar para crear, depurar y publicar aplicaciones Angular.

Los lenguajes que utiliza Angular, y por lo tanto fueron utilizados en esta aplicación del lado del cliente, son los siguientes:

- **HTML5**: es el lenguaje de marcado que se utiliza para el desarrollo de la página web de la aplicación.
- **TypeScript**: es un lenguaje de programación libre y de código abierto desarrollado y mantenido por Microsoft. Es un superconjunto de JavaScript, que esencialmente añade tipado estático y objetos basados en clases. El código fuente del cliente está programado en TypeScript.
- **CSS**: es un lenguaje de diseño gráfico para definir y crear la presentación de un documento en un lenguaje de marcado (ej. HTML). CSS es utilizado para definir el estilo visual de la aplicación web.

⁸ <https://github.com/>

⁹ Angular. Google, Inc. <https://angular.io> 2010-2018

4.1.2 Servidor

El lenguaje de programación utilizado en el servidor fue *Java*. Esta decisión se debe a que este lenguaje fue utilizado a lo largo de la carrera, por lo tanto se evita la etapa de aprendizaje de un nuevo lenguaje.

Viendo la magnitud de este trabajo surge la necesidad de incorporar nuevas herramientas que ayuden a describir el proyecto de software a construir.

En este contexto se decide utilizar *Maven*¹⁰ que es una herramienta de software para la gestión y construcción de proyectos Java. *Maven* brinda soporte para ciertas tareas, por ejemplo la compilación del código, el manejo de dependencias, su empaquetado, la instalación del mismo, etc. También se encarga de especificar el orden de construcción de cada uno de los elementos del proyecto.

Además, surgió la necesidad de incorporar una herramienta que ayude a la creación de la aplicación web y que aborde la complejidad de su configuración. La misma también debe brindar soporte para desplegarla en un servidor, lo que permitiría utilizarla y detectar errores en tiempo de ejecución. Se utilizó para dicha tarea una herramienta llamada *Spring Boot*¹¹, que es un framework muy popular basado en Java para crear aplicaciones web. Proporciona flexibilidad para configurar componentes en múltiples formatos, como XML, *Annotations* y *JavaConfig*.

4.1.3 Herramienta de BI

Las herramientas de Inteligencia de Negocio facilitan la creación, integración e incorporación de analítica visual. En la investigación realizada en [28] se analizan las herramientas de BI más utilizadas, a su vez se buscaron en dicha investigación las herramientas que tengan una versión gratuita para poder utilizar en este trabajo. Algunas de ellas son:

¹⁰ Apache Software Foundation. <http://www.apache.org/> 2002-2015

¹¹ Pivotal Software, Inc. <https://projects.spring.io/spring-boot/>

- Tableau Public¹²
- Sisense¹³
- IBM Watson Analytics¹⁴
- Pentaho Community Edition¹⁵
- Qlik Sense Desktop¹⁶
- Microsoft Power BI¹⁷

Se realizó una capacitación no formal de la herramienta *Qlik Sense Desktop* en la provincia de Buenos Aires. Un ingeniero de sistemas que trabaja con Qlik en una empresa de BI se ofreció a dictar una charla con los aspectos teóricos y prácticos fundamentales de esta herramienta. Gracias a esto se tomó la decisión de utilizar la versión gratuita de esta herramienta.

La plataforma abierta de analítica de datos de Qlik permite crear, integrar e incorporar un marco visual, atractivo y elegante para cualquier dispositivo o app, desde cualquier sitio y en una sola plataforma. Ofrece una completa cartera de soluciones que proporcionan analítica avanzada a todo el abanico de necesidades dentro de la BI. Lo que significa que más personas pueden descubrir conocimientos para crear un mayor valor de negocio.

Dentro de las distintas herramientas que provee Qlik, Qlik Sense permite combinar fácilmente distintas fuentes de datos, independientemente de su tamaño, en una única visualización. El motor asociativo de Qlik indexa cada relación posible de los datos para que se pueda obtener conocimientos inmediatos y explorar en la dirección en que la intuición del usuario lo lleve, ya que el mismo puede realizar subconsultas sobre los datos que está visualizando y estos se adaptan en menos de un segundo [35]. El resultado de la integración de la aplicación creada con Qlik se puede visualizar en el Anexo 2: Aplicación Web.

En la Figura 4.1 se puede ver un ejemplo de la analítica visual que se puede generar con esta herramienta.

¹² <https://public.tableau.com/en-us/s/>

¹³ www.sisense.com

¹⁴ <https://www.ibm.com/watson-analytics>

¹⁵ <https://wiki.pentaho.com/display/COM/Community+Wiki+Home>

¹⁶ <https://www.qlik.com/es-es/products/qlik-sense/desktop>

¹⁷ <https://powerbi.microsoft.com/es-es/>

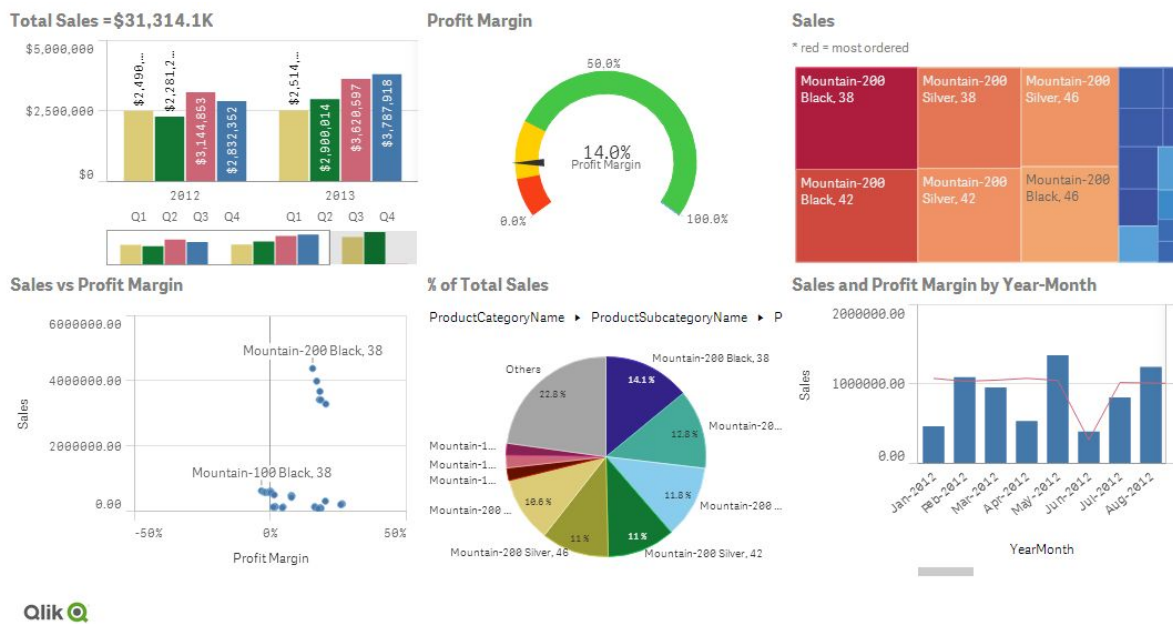


Figura 4.1: Qlik Sense Demo.

4.1.4 Bibliotecas externas

Para lograr una aplicación completa y de buena calidad, fue necesario incorporar diferentes bibliotecas que ofrecen una interfaz bien definida para algunas de las funcionalidades que se necesitan en este proyecto:

- **Log4j:** es una biblioteca open source para Java que permite escribir mensajes de registro, deja constancia de una determinada transacción en tiempo de ejecución [37].
- **Twitter4j:** es una biblioteca no oficial de Java para utilizar la API de Twitter. Con la misma, se pueden integrar fácilmente los servicios de Twitter con cualquier aplicación Java [38].
- **Stanford CoreNLP:** es una biblioteca de procesamiento de lenguaje natural y análisis de sentimientos. Java [21].
- **LingPipe:** es una biblioteca para entrenar clasificadores de sentimientos. Machine Learning [22].

Las bibliotecas para análisis de sentimiento se eligieron de entre varias posibilidades que se encontraron en la web. Los motivos principales de esta selección fueron:

- Son bibliotecas Java.
- Se pueden utilizar de manera gratuita.
- Cuentan con una documentación contundente.
- Incluyen la categoría neutro, otros clasifican solamente en positivo o negativo.
- Los clasificadores se pueden entrenar.

4.2 Etapas del proyecto

Una vez determinadas las tecnologías a utilizar y habiendo definido una aproximación a la arquitectura, se inició con la implementación e integración de dichas tecnologías. Este desarrollo se dividió en tres etapas bien definidas. A continuación se explicará cada una de ellas.

4.2.1 Etapa 1: Prototipo con componentes aislados

Se comenzó investigando posibles herramientas de software y bibliotecas que puedan integrarse para ayudar a resolver las tareas planteadas. A continuación se introducen y detallan los complementos utilizados y la manera en la que fueron implementados de forma aislada.

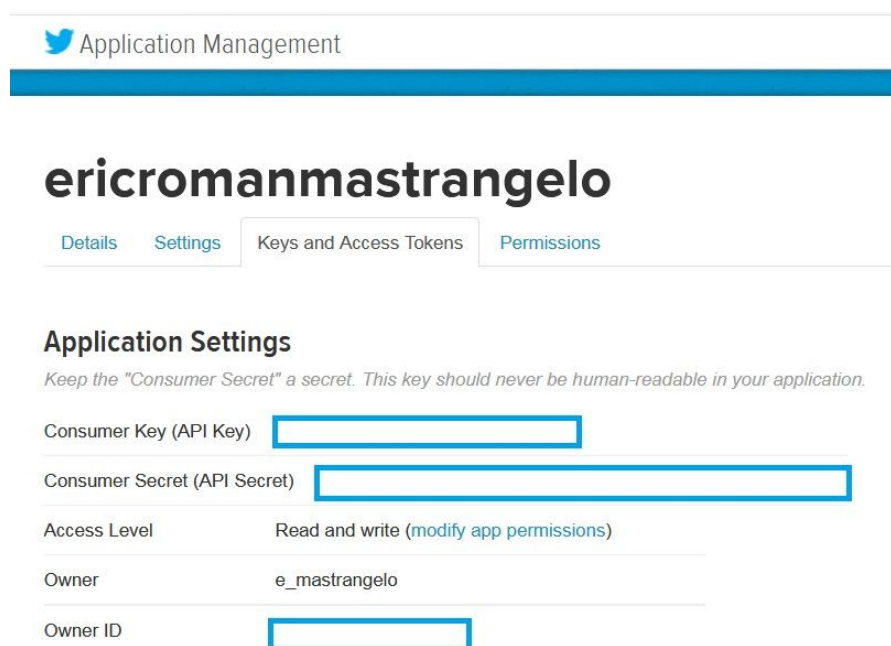
4.2.1.1 Twitter API

Twitter ofrece una interfaz de programación de aplicaciones (API) que facilita la tarea de rastrear y recopilar datos. Con la ayuda de esta API, se pueden

extraer grandes volúmenes de tweets, como así también de perfiles de usuarios y conexiones entre ellos.

Que la API sea de acceso libre, y esté diseñada de forma simple y bien concebida para desarrolladores de terceros brinda una ventaja significativa en contraste al costo de obtener dicha información pública “a mano”. Gracias a esto Twitter ganó una comunidad grande de desarrollos a su alrededor que potenció fuertemente su red social [15].

Twitter permite usar su API de manera gratuita con ciertas restricciones en el volumen de datos consultados cada determinado tiempo. Es decir si se excede el límite de consultas dentro del intervalo temporal, se debe esperar a que el mismo finalice y comience uno nuevo. Para utilizar su API, Twitter obliga a los desarrolladores a registrarse¹⁸ y les brinda claves y tokens de acceso propios. En la Figura 4.2 se muestra un ejemplo de la cuenta privada que deben crear los desarrolladores que desean trabajar con esta API.



Twitter Application Management

ericromanmastrangelo

Details Settings **Keys and Access Tokens** Permissions

Application Settings

Keep the "Consumer Secret" a secret. This key should never be human-readable in your application.

Consumer Key (API Key)

Consumer Secret (API Secret)

Access Level Read and write ([modify app permissions](#))

Owner e_mastrangelo

Owner ID

Figura 4.2: Claves y tokens de acceso para la API de Twitter.

Una vez creado el usuario de Twitter, se incorporó al código fuente del proyecto la biblioteca Twitter4j. La misma permite declarar un objeto de la clase *Twitter* de la siguiente manera:

¹⁸ <https://apps.twitter.com>

```
Twitter twitter = new TwitterFactory().getInstance();
```

Esta línea de código genera una sesión para comunicarse con la API de Twitter, para lograrlo primero revisa las claves y tokens de acceso generadas previamente. Por defecto, estas claves deben ubicarse en la carpeta raíz del proyecto en un archivo de texto llamado *twitter4j.properties*. Este archivo debe contener 4 valores:

consumerKey

consumerSecret

accessToken

accessTokenSecret

Al partir del correcto inicio de la sesión, se puede hacer uso de la API de Twitter libremente. Los métodos de la biblioteca utilizados para obtener información de la misma fueron los siguientes:

- **twitter.getUserTimeline(nombreUsuario, pg):** devuelve los 20 estados más recientes publicados por el usuario dado. Cada estado posee un gran conjunto de atributos, entre ellos: texto del tweet, fecha, ubicación, cantidad de retweets, lenguaje, identificador del estado, etc.
- **twitter.search(query):** Devuelve tweets que coinciden con una consulta especificada. *Query* es una clase de datos que representa una consulta de búsqueda.

Debido a que los textos de los tweets vienen contaminados, se implementó la clase *PreprocesadorTweets*. Este objeto es el encargado de quitar todos los elementos que forman parte de un tweet pero no aportan información, ya que producen alteraciones en los resultados del análisis del sentimiento. El detalle de este proceso se ubica en la sección 5.4: “Preprocesamiento de tweets”.

Tal como se mencionó en capítulos anteriores, existen diferentes formas de obtener los tweets. Esto plantea la necesidad de tres algoritmos que representen las diferentes formas en que el usuario puede seleccionar tweets para analizar. Dichos algoritmos deben ser intercambiables. En este caso se implementó un patrón *Strategy* en donde la clase abstracta fue llamada *SeleccionTweets* y posee tres implementaciones como se observa en la Figura 4.3. El patrón *Strategy* [27] define

una familia de algoritmos, encapsula cada uno de ellos y permite que sean intercambiables. En otras palabras, permite que el algoritmo varíe independientemente de los clientes que lo utilicen.

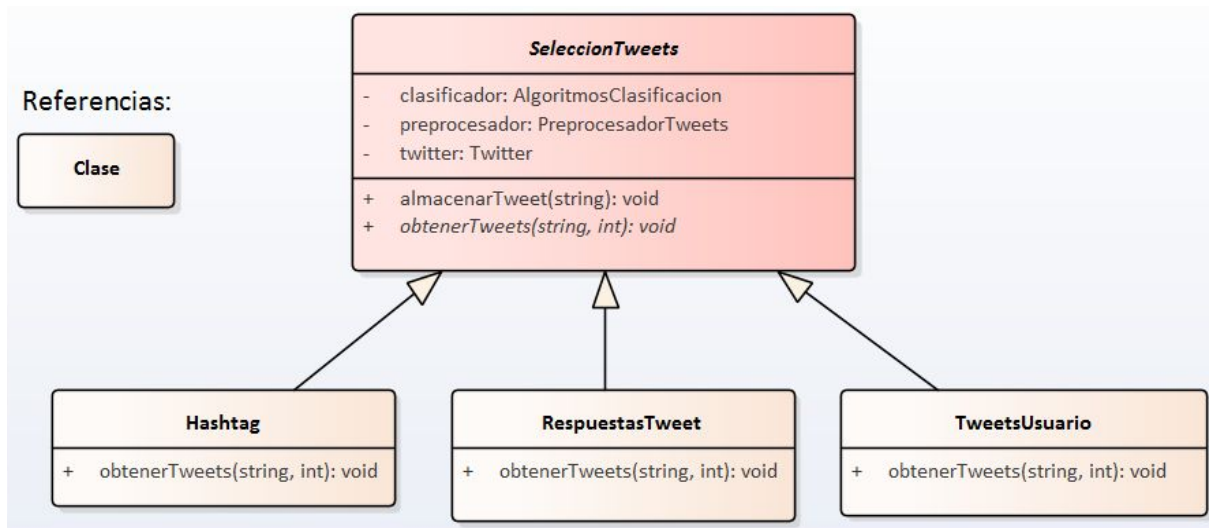


Figura 4.3: Diagrama de clases patrón Strategy.

La clase *SeleccionTweets* es la encargada de obtener los datos de los tweets utilizando la API de Twitter y limpiarlos tal como mencionó anteriormente. También calcula el sentimiento asociado al texto de los mismos, para ello contiene un objeto de la clase *AlgoritmosClasificacion* que puede contener diferentes implementaciones de algoritmos que a su vez realicen diferentes métodos de clasificación (sección 4.2.3.1). De esta manera, *SeleccionTweets* reúne todos los datos de salida necesarios para responder la solicitud del motor de Qlik y que el usuario los pueda visualizar en los distintos gráficos del mismo.

4.2.1.2 Qlik Sense

Una aplicación Qlik consta de un conjunto de gráficos que muestran los datos contenidos en un modelo, el cuál puede ser cargado de diferentes maneras.

El primer paso fue descargar e instalar la versión gratuita de escritorio llamada Qlik Sense Desktop de la página de Qlik¹⁹. Además, se profundizó en la documentación [36] de dicha herramienta con el fin de comprender un poco mejor su

¹⁹ <https://www.qlik.com/us/try-or-buy/download-qlik-sense>

utilidad y funcionamiento. Posteriormente, a la hora de crear la aplicación Qlik, se realizaron las siguientes tareas:

- **Diseñar los Gráficos**

La versión de escritorio contiene un entorno de desarrollo visual e interactivo en donde se diseñan las interfaces y gráficos, pudiendo modificar su ubicación y tamaño. A partir de este diseño, Qlik crea una página web la cuál puede ser accedida mediante cualquier navegador.

- **Cargar el modelo de datos en la aplicación**

Qlik brinda la posibilidad de configurar una gran variedad de fuentes de datos y conexiones a las mismas. En este proyecto la fuente de datos es el servidor de la aplicación, el cual es consultado por Qlik mediante una conexión REST. Qlik administra las conexiones REST desde una interfaz gráfica. En la Figura 4.4 se muestra los parámetros especificados para esta herramienta.

The image displays two side-by-side screenshots of the 'Edit connection (REST)' dialog in Qlik Sense. The left screenshot shows the 'Request' tab with the following fields: URL (http://localhost:8080/keywords), Timeout (2010), Method (GET), Data options (Auto detect response type checked, Key generation strategy: Sequence ID), Authentication (Authentication Schema: Anonymous, Skip server certificate validation unchecked), and Name (RestBackend). The right screenshot shows the 'Query parameters' section with a table of parameters: id (7862654363434), user (nameUser), and cantBajar (2000). It also shows 'Query headers' (empty), 'Pagination' (type: None), and 'Security' (Allow response headers and Allow HTTPS only unchecked). Both screenshots have 'Test Connection', 'Cancel', and 'Save' buttons at the bottom.

Name	Value
id	7862654363434
user	nameUser
cantBajar	2000

Figura 4.4: Conexión rest de aplicación Qlik Sense.

Una vez que el motor consume los resultados del servidor, construye una página web con los gráficos instanciados en dichos resultados.

- **Instanciar los gráficos con los datos**

Una vez cargados los datos en Qlik, cada gráfico nos permite seleccionar un campo del modelo o una función aplicada a un campo para mostrarla en alguna de sus dimensiones. La interfaz gráfica de Qlik facilita en gran medida esta tarea, permitiendo al programador trabajar de manera ágil e intuitiva. A continuación un ejemplo de la configuración de los gráficos en la Figura 4.5.



Figura 4.5: Añadir datos a un gráfico de Qlik Sense.

En este punto, se dejó lista la interfaz de Qlik. En las secciones posteriores se detalla cómo se integró con el servidor de la aplicación para obtener los datos por medio de un servicio REST de forma automática. La aplicación Qlik se almacena en el motor de Qlik bajo el nombre *TesisApp.qvf*.

4.2.1.3 Instanciación de los algoritmos de clasificación

Teniendo en cuenta que el objetivo de este trabajo final es desarrollar una herramienta integral y no investigar e implementar un algoritmo específico para la

clasificación de sentimientos, se tomó la decisión de aplicar un patrón *Strategy* [27]. De esta manera se pueden agregar o quitar algoritmos fácilmente para mejorar la precisión en los resultados de la aplicación.

El diagrama de clases de la implementación de los algoritmos es el de la Figura 4.6.

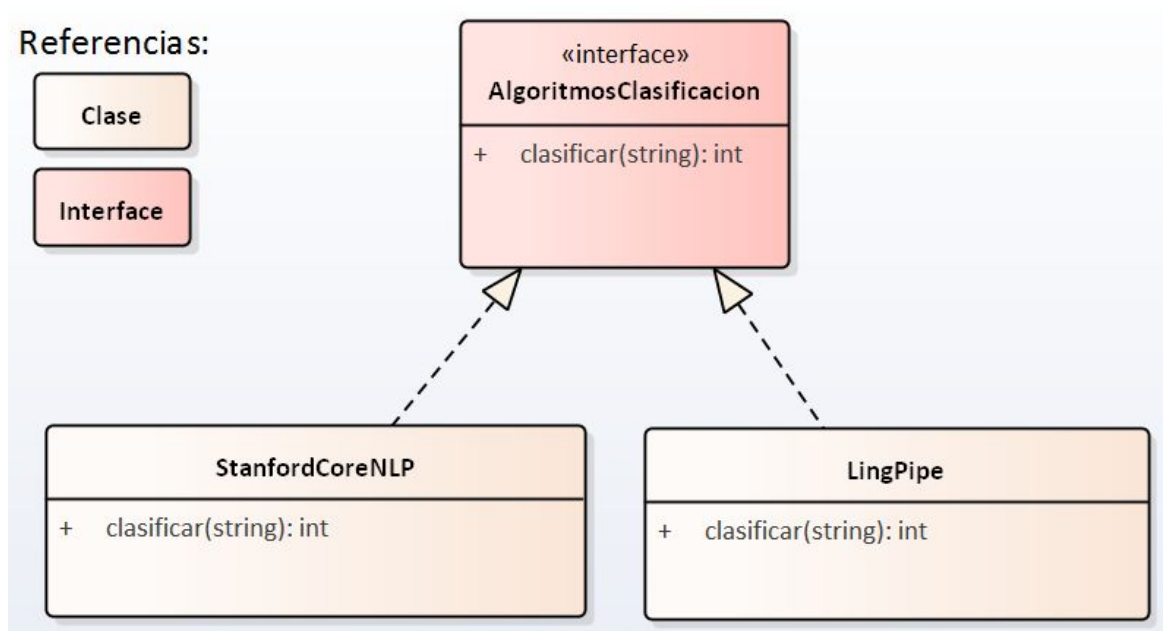


Figura 4.6: Diagrama de clases de algoritmos, patrón Strategy.

Para poder utilizarlos, el primer paso fue acceder al repositorio principal de Maven²⁰ que es el sitio en donde se almacenan los artefactos. Allí se buscaron las versiones específicas de cada una de las bibliotecas²¹ y se agregaron las dependencias correspondientes al archivo de configuración del proyecto.

```
<dependency>
  <groupId>edu.stanford.nlp</groupId>
  <artifactId>stanford-corenlp</artifactId>
  <version>3.8.0</version>
</dependency>

<dependency>
  <groupId>de.julielab</groupId>
  <artifactId>aliasi-lingpipe</artifactId>
```

²⁰ <https://mvnrepository.com/>

²¹ <https://mvnrepository.com/artifact/edu.stanford.nlp/stanford-corenlp/3.8.0>
<https://mvnrepository.com/artifact/de.julielab/aliasi-lingpipe/4.1.0>

<versión>4.1.0</version>
</dependency>

Para integrar ambos algoritmos se implementaron dos clases llamadas *StanfordCoreNLP* y *LingPipe*. Las mismas implementan el método *clasificar*, ambos pseudocódigos se pueden ver en el Anexo 1: algoritmo 1 y algoritmo 2. La función de este método es devolver el sentimiento de un tweet dado. La precondition para que los algoritmos funcionen correctamente es que el texto de entrada esté escrito en el idioma inglés, que es el idioma para el cual fueron entrenados. De lo contrario los datos de salida no tendrán sentido.

4.2.2 Etapa 2: Desarrollo Web

Tal como se menciona en la sección anterior, en principio cada uno de los elementos en esta aplicación estaban aislados y no conformaban ningún tipo de estructura. Al momento de transformar la aplicación en una aplicación web, se debieron tratar diferentes cuestiones relacionadas al cliente (frontend), al servidor (backend), y su integración. A continuación se explicarán cada una de las tareas involucradas en esta etapa.

4.2.2.1 Front end

- Aplicación del lado del cliente: Con la ayuda del framework Angular y mediante la herramienta Angular CLI²² (*command-line interface*), se realizó el desarrollo de la aplicación web del lado del cliente. Con Angular y mediante la línea de comandos se creó el proyecto, es decir, se crearon los archivos en los cuales se realizó la implementación.

\$ng new nombreProyecto //ng = Angular

El paso siguiente fue crear un directorio llamado *components* dentro de la

²² <https://cli.angular.io/>

estructura creada `src/app/`. Luego se ejecutó el siguiente comando para crear un nuevo componente en dicho directorio:

```
$ng g component components/user //componente user creado
```

El mismo crea un nuevo componente con todos los archivos necesarios para el funcionamiento de cualquier aplicación que se ejecutará en un web browser (.ts, .css, .html).

- **Desplegar aplicación en un servidor local:** Una vez creado el cliente, fue necesario ejecutar la aplicación para visualizar las funcionalidades implementadas y poder ver los errores a lo largo del desarrollo. Angular CLI tiene integrado un servidor web, lo que quiere decir que podemos visualizar y usar el proyecto sin necesidad de cualquier otro software. Mediante el comando `$ ng serve` lanzará el servidor web y lo pondrá en marcha. Además, en la terminal se verá como salida del comando la ruta donde el servidor está funcionando. Luego de este paso la aplicación podrá ser accedida mediante el navegador web en el puerto `localhost:4200`.
- **Comunicación con el Backend:** Una vez que la aplicación web del lado del cliente está corriendo en el navegador, la misma interactúa con el usuario quien dispara eventos que se comunican con el servidor. Se implementó desde el cliente una llamada REST para enviar las solicitudes del usuario al servidor. El motivo por el cual se tomó esta decisión es porque en una arquitectura REST cada mensaje HTTP contiene toda la información necesaria para comprender la petición. Como beneficio, ni el cliente ni el servidor necesitan recordar ningún estado de las comunicaciones entre ellos. Para esta tarea fue necesario crear con Angular un nuevo servicio, el comando utilizado fue el siguiente:

```
$ng g service service/data //servicio data creado
```

Dentro de este servicio se realiza la solicitud POST al servidor, el cual corre en el puerto `localhost:8080`. El cuerpo de dicha solicitud es el siguiente:

```
http.post(http://localhost:8080/tweets/solicitudAnalisis?accion=accion&valor
```

=*valor*&*tweetId*=*id*&*cantBajar*=*cant*)

Esta solicitud contiene toda la información necesaria para que el servidor la atienda. A su vez, y a través del *wrapper*, la redirecciona hacia el motor de Qlik mediante el protocolo *WebSocket*. Los parámetros enviados son:

- ***Acción***: representa el tipo de tweets que el usuario quiere obtener. Puede tener los siguientes valores: hashtag (obtiene tweets con cierto hashtag), respuestas (obtiene respuestas a un tweet dado), tweetsusuario (obtiene las publicaciones de un usuario dado).
- ***Id***: identificador de un tweet específico.
- ***Valor***: puede ser el nombre de un usuario o un hashtag.
- ***cantBajar***: cantidad de tweets que se analizarán.

Algunos de los últimos tres parámetros pueden ser nulos, dependiendo la acción elegida.

- **Diseño de la interfaz de usuario**: para dicha tarea se utilizó en primer lugar CSS (Hojas de estilo en cascada). Cada componente creado por Angular contiene un conjunto de archivos, tal como se mencionó anteriormente. Uno de ellos es el archivo CSS, en el cual se diseñaron los botones, títulos, cuadros de textos, imágenes, etc.

En segundo lugar se integraron a la interfaz los gráficos diseñados con la herramienta Qlik. En conjunto, se logró una interfaz robusta e intuitiva.

4.2.2.2 Back end

- **Gestión y construcción del proyecto**: se incorporó la herramienta Maven para describir el proyecto de software a construir. El primer paso fue crear la estructura del proyecto mediante la línea de comandos. Dicha estructura consta de:

- Una ruta de directorios para el código fuente de la aplicación
src/main/java
 - Un archivo llamado *pom.xml*: contiene el Modelo de Objetos del Proyecto (POM), es decir contiene toda la información importante sobre el proyecto. Este archivo es la unidad básica de trabajo en Maven, lo cual es muy importante ya que en Maven todo gira en torno a la noción de proyecto.
- **Resolver dependencias:** todas las bibliotecas y herramientas externas utilizadas en esta aplicación fueron importadas desde un repositorio remoto a un repositorio local con la ayuda de Maven. Para ello fue necesario agregar cada dependencia al archivo *POM.xml* antes mencionado, de la siguiente manera:

```
<dependency>  
  <groupId>nombreEmpresa</groupId>  
  <artifactId>nombreConcretoDelPaquete</artifactId>  
  <version>numeroVersion</version>  
</dependency>
```

Las mismas se empaquetan de forma automática junto con la aplicación para el correcto funcionamiento de la herramienta.

- **Desplegar aplicación en un servidor local:** se utilizó el framework Spring boot, el cual contiene un artefacto llamado *spring-boot-starter-web*. Para incorporar dicho framework simplemente se agrego la siguiente dependencia al archivo *POM.xml* del proyecto:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```


Este artefacto levanta un servidor web autónomo embebido de forma local. Por defecto, al ejecutarse la misma reside en el puerto *localhost:8080*, permitiendo utilizar y testear la aplicación de manera simple y rápida.

- **API REST del Servidor:** se implementó en el servidor un servicio REST, el cual es una interfaz que utiliza directamente el protocolo HTTP para obtener datos o indicar la ejecución de operaciones sobre los datos. Para esto, se utilizó el framework Spring Web. Spring utiliza un enfoque Model-View-Controller (MVC) para construir servicios web REST, por lo que las solicitudes HTTP son manejadas por un controlador. Estos componentes se identifican fácilmente mediante la anotación *@RestController*. En esta aplicación el servidor contiene cuatro clases controladoras que se encuentran a la espera de una solicitud, en el puerto *localhost:8080*. Las solicitudes provenientes del motor de Qlik son para analizar tweets que según los parámetros pueden ser de diferentes tipos:

- Respuestas a un tweet dado

http.get(http://localhost:8080/tweets/respuestas?id=numID&valor=usuario&cantBajar=cant)

- Tweets de un usuario

http.get(http://localhost:8080/tweets/tweetsusuario?valor=usuario&cantBajar=cant)

- Tweets que contengan cierto hashtag

http.get(http://localhost:8080/tweets/hashtag?valor=hashtag&cantBajar=cant)

Las solicitudes provenientes del cliente son para redireccionar al motor de Qlik y que este obtenga los resultados (mediante cualquiera de los tres llamados anteriores) y actualice sus gráficos. La solicitud del cliente es la siguiente:

http.post(http://localhost:8080/tweets/solicitudAnalisis?accion=accion&valor=valor&tweetId=id&cantBajar=cant)

- **Registro de ejecución del servidor:** Como en toda aplicación, es de vital importancia contar con mecanismos de seguimiento de actividad, lo que tiene como consecuencia un ahorro en el tiempo necesario para determinar las causas de ciertos errores que se detectan durante la ejecución de las mismas. Se utilizó la biblioteca Log4j para escribir mensajes de registro, con el propósito de dejar constancia de todas las transacciones en tiempo de ejecución. Log4j permite filtrar los mensajes en función de su importancia.

4.2.3 Etapa 3: Integración Qlik Web

La API del motor de Qlik utiliza mensajes JSON enviados mediante un WebSocket para pasar información entre el motor Qlik Sense y los clientes.

La API del motor de Qlik consta de un conjunto de objetos que representan aplicaciones, listas, gráficos, etc. La forma de realizar modificaciones sobre estos objetos es enviando solicitudes a dicha API.

Dicho esto y habiendo creado la aplicación Qlik con los gráficos necesarios, se decidió aplicar el patrón Adapter [27] y crear un componente intermedio entre el cliente y el motor de Qlik.

El patrón Adapter permite que dos clases con interfaces incompatibles trabajen juntas. Básicamente su función es convertir la interfaz de una clase en otra interfaz. El elemento que realiza de intermediario entre dichas clases se denomina Wrapper [27]. A la hora de implementarlo se analizaron ventajas y desventajas de la ubicación que tendría dentro de la arquitectura del sistema, ya sea del lado del cliente o del servidor. Se tomó la decisión de implementarlo del lado del servidor debido a los siguientes motivos:

- **Seguridad:** Según [29] se debe restringir el uso de WebSocket en los navegadores web ya que se registran varios problemas de seguridad. Esto se debe a que el protocolo WebSocket aún no está estandarizado.
- **Compatibilidad:** No todos los navegadores web tienen compatibilidad con el protocolo WebSocket [30].
- **Bloqueo por proxys:** La mayoría de las redes corporativas poseen servidores proxy que median las conexiones HTTP. Algunos de estos

servidores cancelan las conexiones WebSocket para prevenir ataques maliciosos [31].

Su función es convertir las peticiones REST del cliente en un mensaje WebSocket y enviarlo al motor de Qlik. Este componente intermedio se integró a la aplicación tal como se ve en la Figura 4.7.

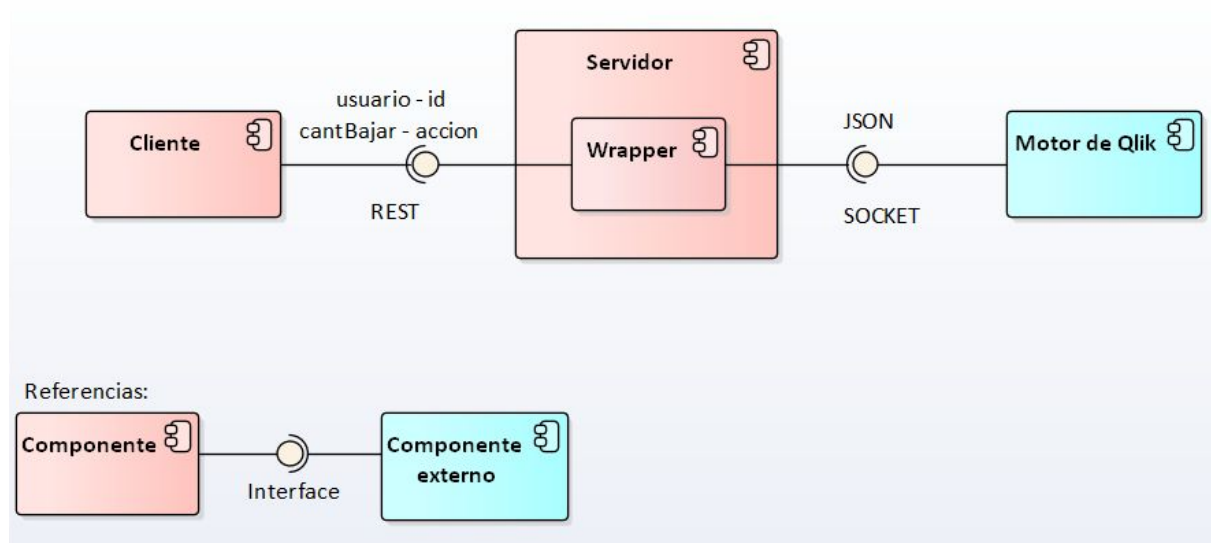


Figura 4.7: Componente intermedio entre cliente-servidor.

Básicamente, el cliente le pasa al *Wrapper* los parámetros correspondientes a los tweets que desea obtener para analizar por medio de un llamado REST. El *Wrapper* crea un canal de comunicación bidireccional (WebSocket) con el motor de Qlik de la siguiente manera: `ws://localhost:4848/app/SentimentToolApp.qvf`. Luego, le envía por el canal cuatro mensajes²³ en formato JSON para que ejecute la acción indicada por el usuario. Estos mensajes son:

- **“OpenDoc”**: el motor de Qlik contiene todas las aplicaciones creadas con Qlik Sense Desktop. El *Wrapper* debe especificar qué aplicación desea modificar, en este caso *SentimentTool.qvf*.
- **“ModifyConnection”**: tal como se explicó anteriormente, Qlik hace un llamado REST al servidor de la aplicación para cargar en sus gráficos los tweets obtenidos. Este método modifica los parámetros del llamado REST, añadiendo los nuevos parámetros que solicitó el usuario.

²³ <https://help.qlik.com/en-US/sense-developer/February2018/apis/EngineAPI/services-Global-OpenDoc.html>

- **“DoReload”**: indica a Qlik Sense que debe cargar sus datos. Al ser su fuente un servidor, y su conexión un llamado REST al mismo, Qlik pide los nuevos datos para mostrarlos en los distintos gráficos.
- **“DoSave”**: guarda los cambios realizados en la aplicación *SentimentTool.qvf*. La ejecución de este método es el que recarga efectivamente los datos de salida en los gráficos que visualiza el usuario.

El último paso fue añadir la interfaz gráfica creada con Qlik a la interfaz de usuario del cliente. Como en toda aplicación web el cliente cuenta con un archivo en lenguaje HTML que sirve como interfaz entre la aplicación y el usuario final. Para integrar la página web generada por Qlik con la interfaz web del cliente se utilizó un *iframe*. El mismo permite insertar un archivo HTML dentro del HTML principal, más específicamente, permite embeber los gráficos de la herramienta Qlik en el HTML del cliente. La interfaz de usuario final se puede ver en la Figura 4.8.

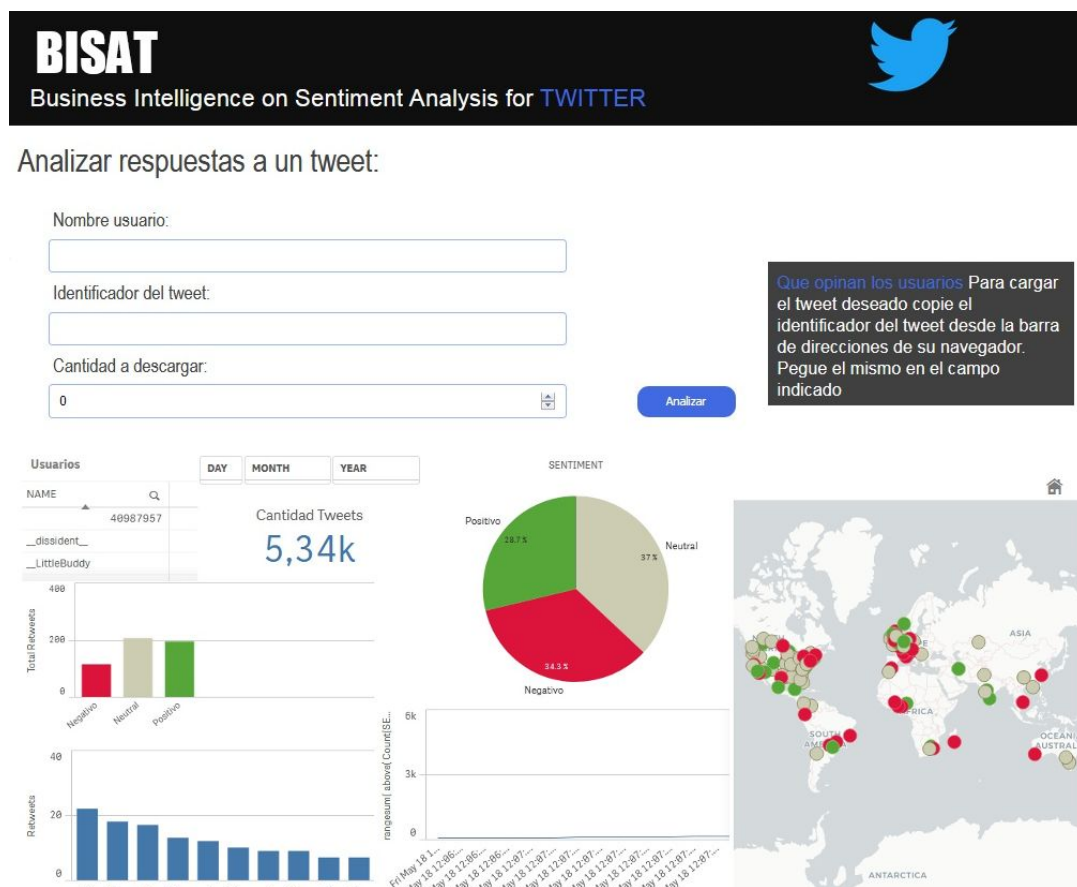


Figura 4.8: Interfaz de usuario.

4.3 Arquitectura final del sistema

En esta sección se detalla el diseño arquitectónico final del sistema. La misma, se divide en dos subsecciones, la primera dedicada a las vistas estáticas de los componentes del sistema, y la segunda a las vistas dinámicas que definen la comunicación entre los mismos.

4.3.1 Vistas estáticas del Sistema

Tal como se menciona en el capítulo tres: Solución Propuesta, la arquitectura de esta aplicación es la que adoptan todas las aplicaciones web: Modelo Cliente-Servidor. A continuación se aumentará el nivel de detalle de cada uno de sus componentes.

Servidor

El servidor de esta aplicación consume y brinda servicios a dos herramientas externas tal como se ve en la Figura 4.9. Una de las herramientas es la fuente de datos para analizar (Twitter), y la otra es el motor de la aplicación de BI con la cual se visualizan los resultados (Qlik). A su vez se creó un componente intermedio para la comunicación con esta última.

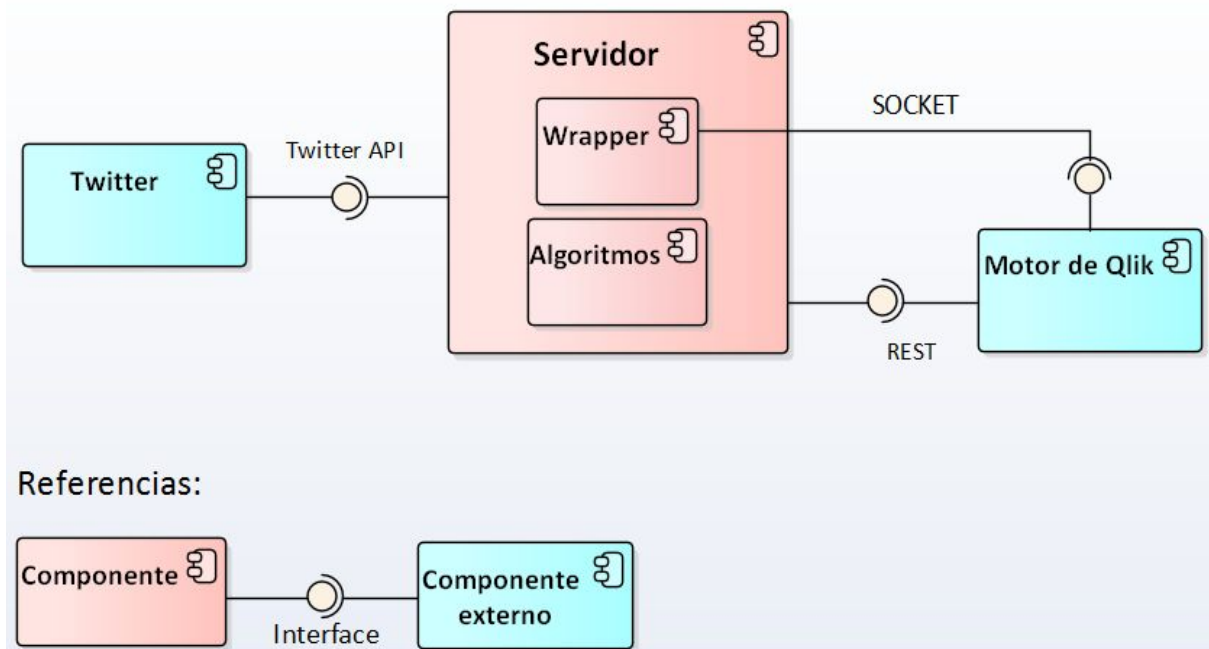


Figura 4.9: Diagrama de componentes del servidor.

Tal como se explica en la sección 3.2.3, se tomó la decisión de aplicar el patrón *Adapter* implementando un *Wrapper* como intermediario entre las peticiones que realiza el cliente al servidor. Su función es convertir las peticiones REST del cliente en un mensaje WebSocket y enviarlo al motor de Qlik. Este componente intermedio se integró a la aplicación tal como se ve en la Figura 4.10.

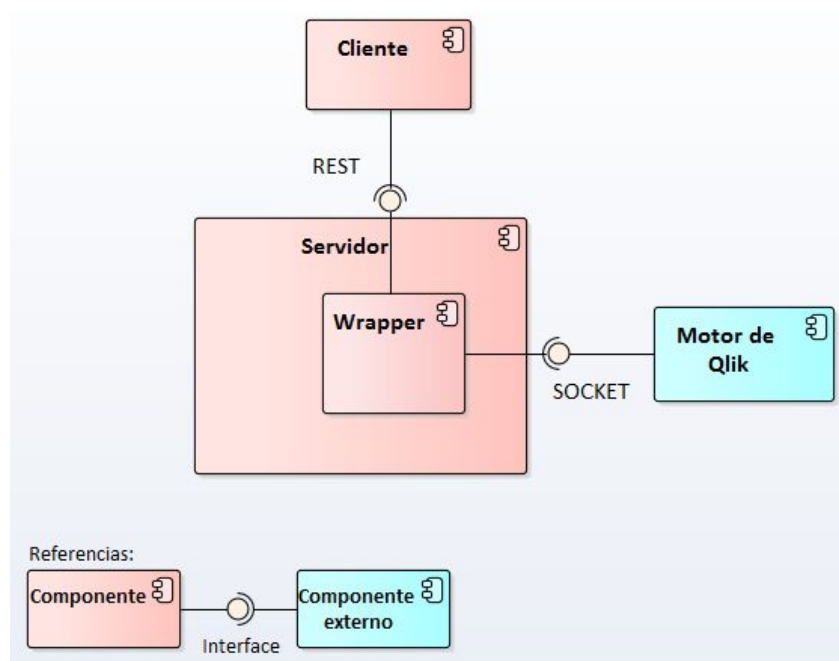


Figura 4.10: Patrón Adapter Cliente-Servidor.

El servidor de la aplicación propiamente dicha, es el encargado de obtener los tweets mediante la Api de Twitter, clasificarlos y enviar la información de salida en formato CSV (archivo separado por comas) para que sea interpretada por el motor de Qlik. A su vez, mediante el wrapper, envía la orden al motor de Qlik para que este recargue los gráficos con los resultados y se los muestre al usuario. En la Figura 4.11 se muestran los paquetes principales que conforman el servidor.

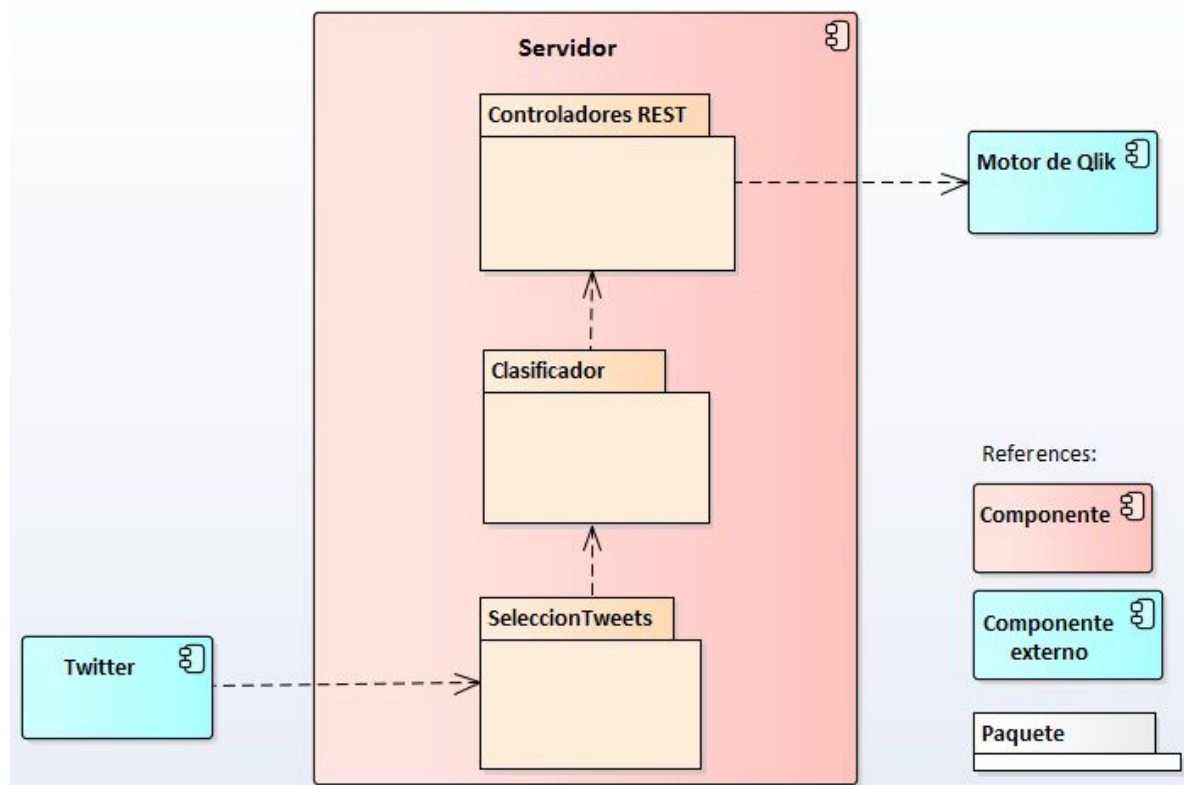


Figura 4.11: Diagrama de paquetes del servidor.

Cliente

El cliente fue implementado con la ayuda del framework Angular. El mismo está alojado en el navegador web de cada usuario y está compuesto por los siguientes artefactos:

- **Componente usuario:** es el artefacto principal en el cliente. Aloja el archivo HTML con la interfaz de usuario, el archivo CSS que da estilo a dicho HTML y el archivo typescript que contiene la lógica del cliente.
- **Servicio REST:** es el proveedor de datos, en él se delega la responsabilidad de acceder a la información y las operaciones con la misma.

Como en toda aplicación web el archivo en lenguaje HTML sirve como interfaz entre la aplicación y el usuario final. Para integrar la página web generada por Qlik con la interfaz web del cliente se utilizó un *iframe*. Las peticiones del cliente son enviadas al servidor a través del servicio mencionado mediante el protocolo REST, tal como se ve en la Figura 4.12.

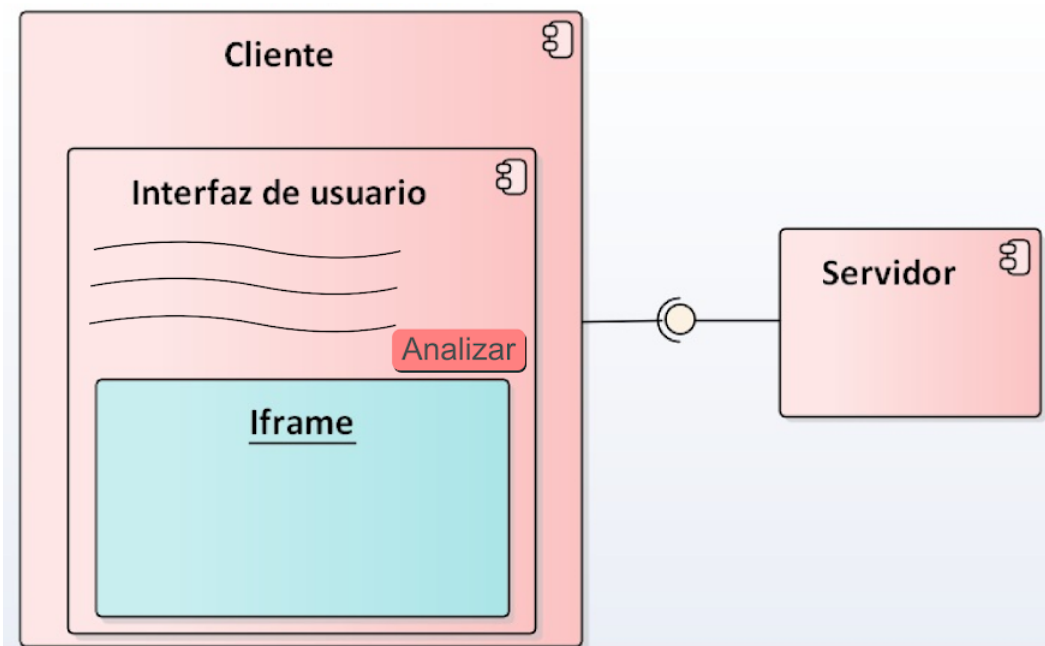


Figura 4.12: Diagrama de componentes del cliente.

En la Figura 4.13 se muestra el diagrama final de la arquitectura de la aplicación.

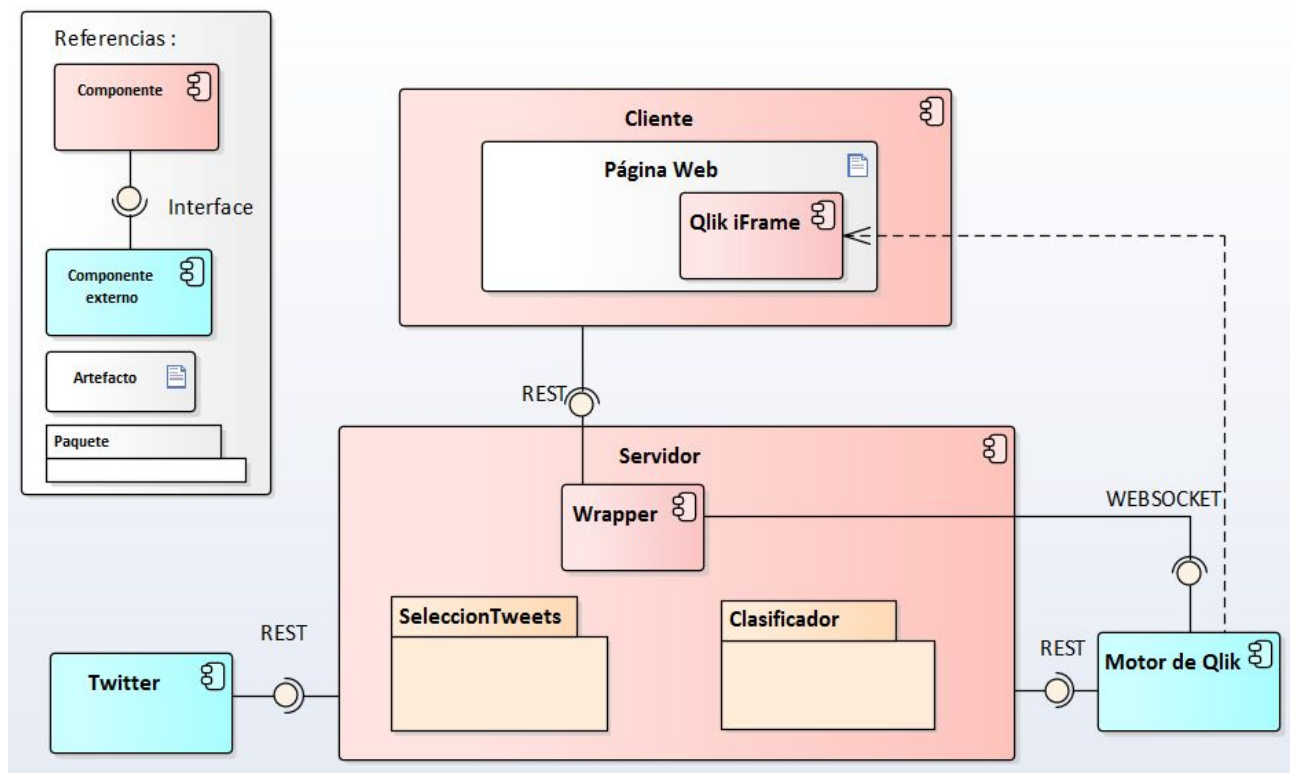


Figura 4.13: Arquitectura cliente-servidor de la aplicación.

4.3.2 Vistas dinámicas del Sistema

El flujo básico. de la aplicación se puede visualizar en el diagrama de actividades de la Figura 4.14.

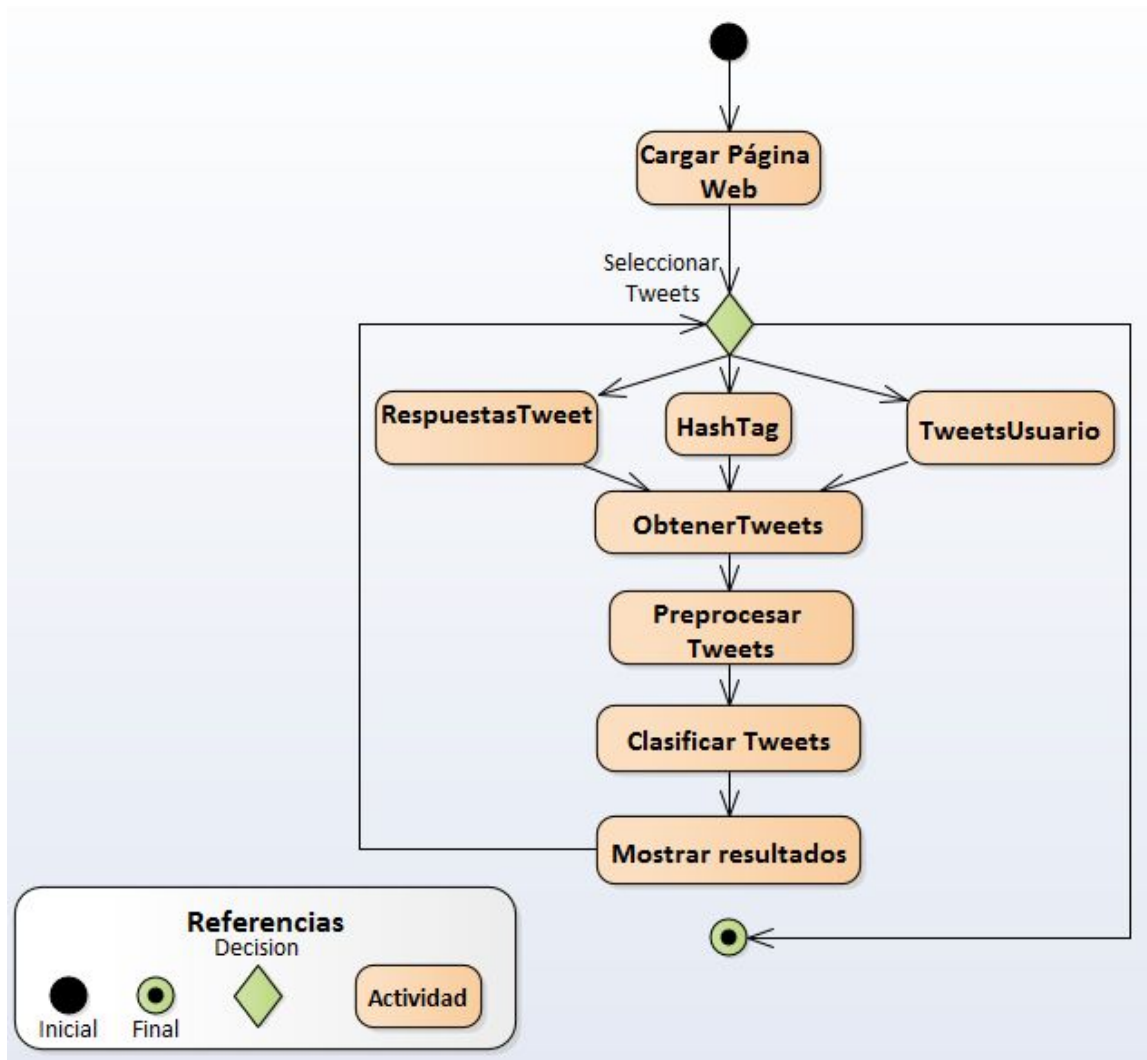


Figura 4.14: Diagrama de actividades de la aplicación.

El *workflow* (flujo de trabajo) de la aplicación consta de tres acciones posibles que se distinguen solamente por el tipo de tweets que obtienen para analizar. Al margen de esto, las tareas que realiza la aplicación son las mismas para cualquiera de las tres opciones.

El motor de Qlik aloja distintas aplicaciones creadas con la herramienta Qlik Sense, motivo por el cual es necesario enviar los mensajes que se visualizan en la Figura 4.15 para indicar el identificador de la aplicación que se desea utilizar y los parámetros que necesita dicha aplicación para pedir los datos al servidor.

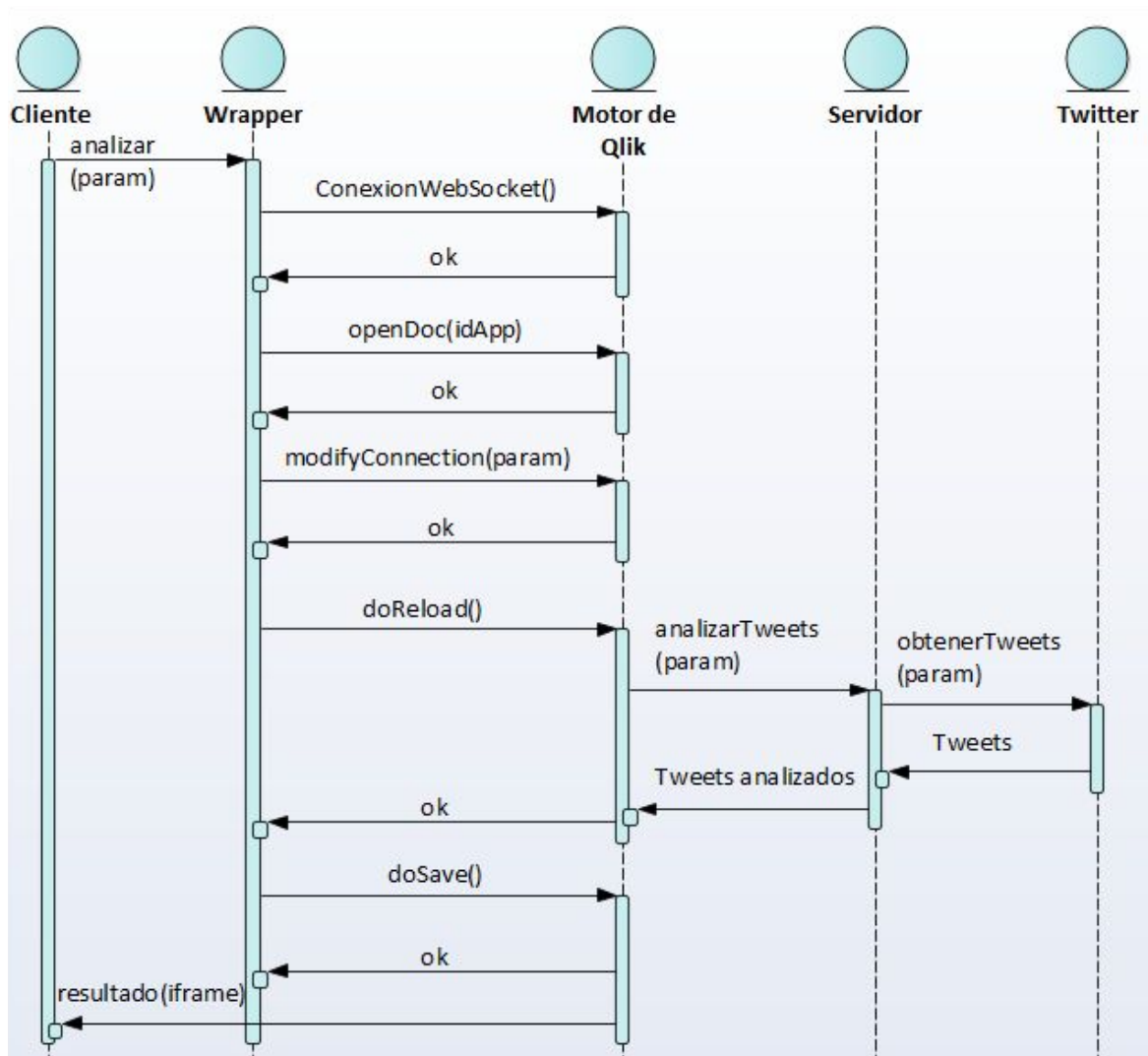


Figura 4.15: Diagrama de secuencia Cliente-Servidor

Con el mensaje *doReload()* visto en la Figura 4.15, el motor de Qlik solicita al servidor de la aplicación toda la información necesaria para graficar la salida. Para ello, a su vez, el servidor debe hacer uso de la Api de Twitter para obtener los datos. Posteriormente los procesa y retorna los resultados de la solicitud, automáticamente el usuario podrá ver dichos datos en su navegador web.

Capítulo 5

Metodología

En este capítulo se describirán la metodología y los artefactos involucrados durante el desarrollo y evaluación de la herramienta. Se detalla de qué manera se trabajó tanto con los algoritmos como con las bases de datos y de qué forma se analizaron los resultados obtenidos con los mismos.

5.1 Algoritmos utilizados

Para este proyecto se investigaron e integraron dos algoritmos de clasificación de sentimientos que utilizan técnicas de machine learning, uno previamente entrenado y otro entrenado específicamente para esta aplicación. Los mismos se encargan de clasificar un texto dado (en el idioma inglés) en tres categorías: positivos, neutros o negativos. A continuación se detalla de qué manera fueron utilizados.

5.1.1 Stanford CoreNLP

Este algoritmo no requiere entrenamiento previo para su utilización en esta herramienta. Tal como se explicó en el Marco teórico de este proyecto, clasifica el texto de entrada en cinco posibles categorías: muy negativo, negativo, neutro, positivo, muy positivo. Para ello estima la probabilidad de que el texto pertenezca a cada categoría, dicha probabilidad es la confianza.

Dado que la herramienta desarrollada en este proyecto clasifica el sentimiento en tres categorías (negativo, neutro, positivo) se interpreta la salida de este algoritmo de la siguiente manera.

- Muy negativo = $P(--)$
 - Negativo = $P(-)$
 - Neutro = $P(n)$
 - Positivo = $P(+)$
 - Muy Positivo = $P(++)$
- $\left. \begin{array}{l} \text{Muy negativo} \\ \text{Negativo} \end{array} \right\} \text{Negativo} = P(--)+P(-)$
 $\left. \begin{array}{l} \text{Neutro} \end{array} \right\} \text{Neutro} = P(n)$
 $\left. \begin{array}{l} \text{Positivo} \\ \text{Muy Positivo} \end{array} \right\} \text{Positivo} = P(++)+P(+)$

Es decir, si el texto es clasificado en la categoría muy negativo o negativo, el resultado es la categoría negativo y la confianza es la suma de las confianzas de ambas categorías. Las categorías muy positivo y positivo son tratadas de igual manera, resultando en la categoría positivo con la suma de las confianzas asociadas.

5.1.2 LingPipe

Este algoritmo requiere un entrenamiento previo para su utilización. Para dicha tarea, el primer paso fue conseguir una base de datos con la cual realizar el entrenamiento y luego poder validar el clasificador obtenido. El detalle de la búsqueda y procesamiento de la misma se encuentra en la sección siguiente.

Una vez elegidas las dos bases de datos, el algoritmo fue entrenado con cada una de ellas para clasificar el sentimiento en tres diferentes categorías: positivos, neutros y negativos. Para ello, los textos que conforman dicha base de datos fueron colocados en tres archivos independientes dentro de una carpeta según su categoría, tal como se ve en la Figura 5.1.

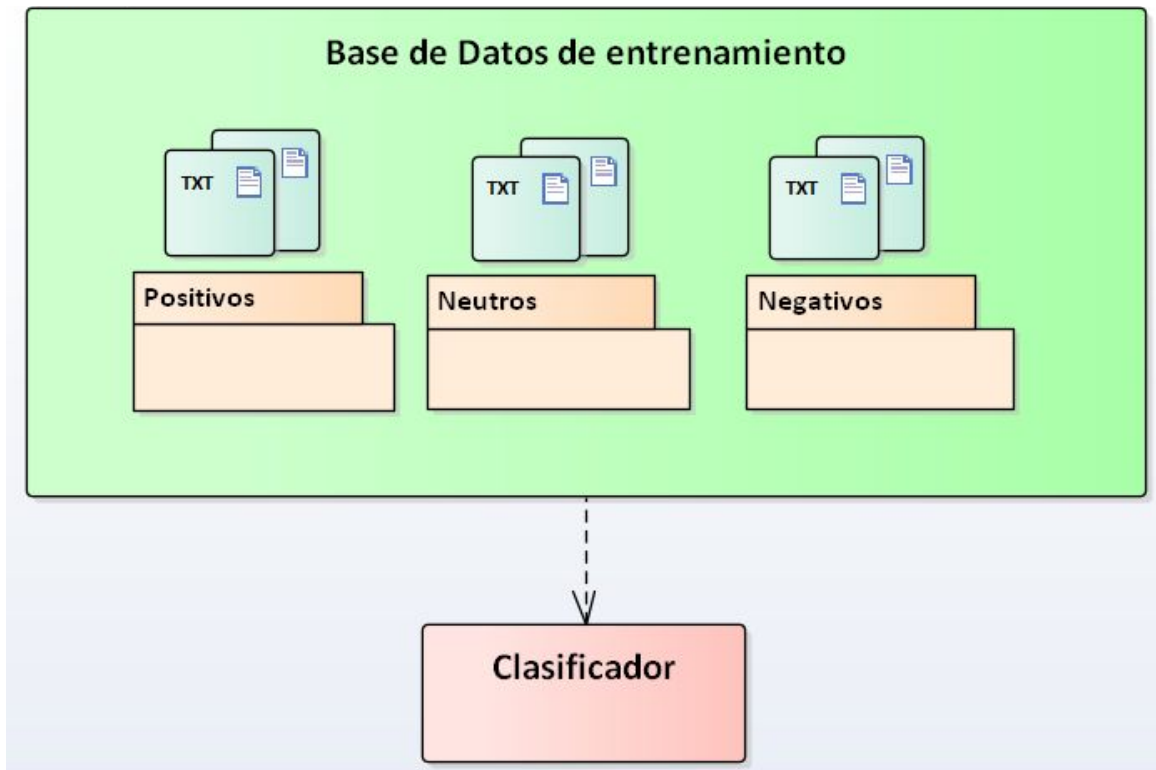


Figura 5.1: Esquema de ubicación de datos de entrenamiento.

Luego, se implementó una clase llamada *Entrenador* que contiene un método llamado *entrenar*. Este método se ejecuta a través de las categorías, recorriendo y procesando los archivos de entrenamiento que encuentra en cada una de ellas. El texto leído de los archivos, es utilizado para entrenar al clasificador para la categoría especificada. El resultado es un archivo que almacena la configuración del clasificador. Dicho archivo contiene un modelo de lenguaje para cada una de las categorías. La implementación de este método fue sacada de la página oficial en la sección *demos/tutorial/sentiment-analysis* [22], el pseudocódigo del mismo puede verse en el Anexo 1: algoritmo 3.

Los dos algoritmos resultantes se denominan LingPipe Palabras y LingPipe Tweets.

5.2 Base de Datos de entrenamiento y validación

En esta sección se detalla el método de selección y preprocesamiento que se aplicó en la base de datos de entrenamiento. La misma no solo fué utilizada para entrenar los algoritmos, sino que fue utilizada también para obtener métricas de los algoritmos resultantes.

El primer paso fue conseguir una base de datos para dichas tareas. La misma debe no solo contener textos en inglés, sino que cada una de sus entradas debe estar clasificada según el sentimiento asociado al texto que la compone. Se realizó una búsqueda en la web y se estudiaron diferentes alternativas, las mismas se muestran a continuación en la Tabla 5.1.

Base de datos	Características
Palabras ²⁴	La base de datos consiste en 3 archivos: uno de palabras positivas, otro de palabras neutrales y el último con palabras negativas
Opiniones de películas ²⁵	Opiniones de distintas películas clasificadas en positivas y negativas
Tweets sobre un tema específico ²⁶	Se encontraron varias bases de datos con tweets relacionados a un mismo tema. Por ejemplo “nuclear energy”, “Airlines”, entre otras
Texto de tweets ²⁷	Textos de tweets sin ninguna relación entre ellos. Clasificados en 13

²⁴ http://mpqa.cs.pitt.edu/lexicons/subj_lexicon/

²⁵ <http://ai.stanford.edu/~amaas/data/sentiment/>

²⁶ <https://www.figure-eight.com/data-for-everyone/>

²⁷ <https://www.figure-eight.com/data-for-everyone/>

	categorías según el contenido emocional (como la felicidad, la tristeza, la ira, etc).
--	--

Tabla 5.1: Tabla comparativa bases de datos encontradas.

De las opciones estudiadas se descartaron dos de ellas debido al sesgo hacia un tema determinado: reseñas de películas y tweets sobre un tema específico. Las dos bases de datos restantes fueron utilizadas en este proyecto, las mismas son: la de palabras y la de textos de tweets.

Ambas fueron divididas de la siguiente manera, dejando la porción mayor de las mismas destinada para el entrenamiento y la otra para validar que tan buenos son los algoritmos entrenados obtenidos.

75% Datos de entrenamiento	25% Datos validación
--------------------------------------	--------------------------------

Luego de entrenar el algoritmo de la biblioteca LingPipe con cada una de ellas, se realizaron experimentos para medir su rendimiento con diferentes métricas. Los cálculos se realizaron clasificando tweets en positivos, neutros y negativos. Dichos resultados se detallan en el capítulo seis: Resultados-Métricas de Algoritmos.

La base de datos seleccionada para esta herramienta es la de textos de tweets y fue descargada de *Data for everyone* [20], un sitio de bases de datos públicas. Cada una de sus entradas consta de el texto de un tweet y está clasificada en 13 distintos grupos. Cada grupo representa una emoción: vacío, tristeza, entusiasmo, neutral, preocupación, sorpresa, amor, diversión, odio, felicidad, alivio, enfado, aburrimiento. Con el fin de acotar las categorías en positivo, neutro y negativo se agruparon las 13 categorías antes mencionadas en:

- Positivos: entusiasmo, amor, diversión, felicidad, alivio.
- Neutros: neutral, sorpresa.
- Negativos: vacío, tristeza, preocupación, odio, enfado, aburrimiento.

Las tres nuevas categorías contienen 6400 entradas cada una, conformando una base de datos con casi 20 mil entradas en total.

5.3 Métricas de Algoritmos

Tal como se explicó en la primer sección de este capítulo, se implementaron tres algoritmos para realizar la clasificación del sentimiento de un tweet en positivo, neutro o negativo. Dichos algoritmos tienen las siguientes características:

- **Stanford CoreNLP**: algoritmo externo, integrado y listo para utilizar.
- **LingPipe Palabras**: algoritmo entrenado con una base de datos de palabras.
- **LingPipe Tweets**: algoritmo entrenado con una base de datos de tweets.

Se calcularon diferentes métricas con el fin de determinar qué algoritmo es el que mejor se adapta a esta aplicación. A continuación se introducirán y detallarán cada una de ellas.

Precisión

En reconocimiento de patrones, *precisión* (también llamada valor positivo predicho) es la fracción de instancias relevantes entre las instancias recuperadas. Esta métrica intenta responder la siguiente pregunta: ¿Qué proporción de identificaciones positivas fue correcta?

La precisión se define de la siguiente manera:

$$\text{Precisión} = \frac{\text{VP}}{\text{VP} + \text{FP}}$$

Donde VP son los verdaderos positivos y FP son los falsos positivos.

Recall

Recall (también conocida como sensibilidad) es la fracción de instancias relevantes que se han recuperado sobre la cantidad total de instancias relevantes. Esta métrica intenta responder la siguiente pregunta: ¿Qué proporción de positivos reales se identificó correctamente?

La misma se define de la siguiente manera:

$$\text{Exhaustividad} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

Donde VP son los verdaderos positivos y FN son los falsos negativos.

Performance

En informática, la performance es una medida o cuantificación de la velocidad con que se realiza una tarea o proceso. Para calcular dicha métrica se recopilaban 5000 tweets al azar. Los mismos fueron clasificados por cada uno de los algoritmos estudiados en igualdad de condiciones. La performance se evaluó en cuanto al tiempo exacto que tardaron en obtener los resultados.

Confianza

Además, se analizaron métricas en base a la confianza que retorna cada algoritmo asociada a su respuesta. El primer cálculo realizado fue el de la distribución de dicha confianza, lo cual permite conocer qué tanta confianza tiene cada algoritmo en el resultado que retorna. El proceso realizado consistió en contar, para cada valor de confianza, la cantidad de veces que el algoritmo lo retorna. Es decir, si dado un texto el algoritmo lo clasifica en determinada categoría con un X% de confianza, se le suma uno a X.

Por otro lado se analizó la exactitud asociada a la confianza, es decir que porcentaje de aciertos tiene cada algoritmo en cada valor de confianza posible. Esto

permite saber, por ejemplo, si un algoritmo acierta mucho o poco con un 99% de confianza.

Para ambas métricas cada algoritmo analizó de manera automatizada 4800 tweets, 1600 de cada categoría posible.

5.4 Preprocesamiento de tweets

Los tweets en general, contienen diversos elementos que forman parte del texto del mismo. Algunos de ellos son de utilidad para la predicción del sentimiento y otros no. En la Figura 5.2 se ve un ejemplo de un tweet.



Figura 5.2: Ejemplo de un tweet.

Una vez obtenido el tweet completo con la Api de Twitter, si se extrae el texto asociado al mismo, el resultado sería el siguiente:

🏁STAGE 2 - BIKES 🏁

*Provisional rankings

1. @kmbenavides 3h14'58"

2. @quintanilla102 +2'16"

3. @tobyprice87 +3'41"

#Merzouga2018

Los siguientes elementos del tweet no son de utilidad: emoticones de banderas a cuadros, menciones a otros usuarios (@nombre), símbolos (+, ', ", ., *) y números. Con el fin de mejorar la calidad de la predicción del sentimiento se desarrolló la clase *PreprocesadorTweets*. El constructor de esta clase recibe dos parámetros. Uno de ellos es el texto del tweet original y el otro es una expresión regular. Su función es detectar cadenas de caracteres a partir de la expresión y eliminarlas. Las expresiones regulares reconocen los elementos mostrados en la Tabla 5.2.

Elemento	Motivo
Salto de línea "\n"	Al momento de almacenar los tweets se separan por saltos de línea.
Encabezado Retweet "RT"	Texto agregado por la red social, no pertenece al texto escrito por el usuario
Emoticones	El código de los emoticones contamina el texto del tweet
Dirección e-mail	No aporta información, y empeora la predicción del sentimiento
Dirección URL	No aporta información, y empeora la predicción del sentimiento
Mención @nombreUser	No aporta información, y empeora la predicción del sentimiento
Números	No aporta información, y empeora la predicción del sentimiento
Símbolos y letras sueltas	No aporta información, y empeora la predicción del sentimiento
Ordinales "st-nd-rd..."	No aporta información, y empeora la predicción del sentimiento

Tabla 5.2: Tabla de elementos reconocidos y extraídos de un tweet.

Tal como se explicó en capítulos anteriores, otro elemento que se encuentra en la gran mayoría de los tweets es el hashtag. Los mismos están compuestos por palabras y números. Surgió la idea de utilizar las palabras como parte del texto a clasificar, para ello se realizó una investigación de qué tipos de hashtag se utilizan mayormente y qué tanto aportan al sentimiento de la publicación. Para ello, se contabilizaron a mano 100 tweets elegidos al azar, los mismos fueron clasificados según la forma en que delimitan las palabras. Luego de dicho experimento se conformaron nueve conjuntos de tweets. A continuación se muestran los resultados en la Tabla 5.3.

Hashtag	Descripción	Frecuencia	Se puede detectar
#PALABRA	palabra en mayúscula	9%	si
#palabra	palabra en minúscula	6%	si
#Palabra	Palabra que comienza con mayúscula	20%	si
#PalPal	Concatenación de palabras que empiezan con mayúscula	47%	si
#palPal	Concatenación de palabras que empiezan con mayúscula, menos la primera	6%	si
#pal1pal2	Concatenación de palabras en minúscula	1%	no
#PAL1PAL2	Concatenación de palabras en mayúscula	1%	no
#palPAL	Concatenación de palabras en mayúscula y minúscula	5%	si
#PALPal	Concatenación de palabras, donde la primer letra de una de ellas se pierde	5%	no

Tabla 5.3: Investigación de tipos de Hashtags y su frecuencia.

Se tomó la decisión de no eliminar los hashtags ya que el 93% están compuestos por palabras que se pueden detectar. Esta tarea se realizó con un método de la clase

PreprocesadorTweets llamado *separarHashtag*, que por medio de una expresión regular detecta y separa las palabras con un espacio en blanco. Por ejemplo, traduce #ArgentinaEstamosConVos a Argentina Estamos Con Vos.

Luego de este preprocesamiento, el texto puro de los tweets obtenidos queda listo para su posterior uso. Cada uno de ellos es clasificado individualmente por los algoritmos y luego es almacenado junto con el sentimiento asociado.

Capítulo 6

Resultados

En este capítulo se presentan los resultados obtenidos a lo largo de este proyecto. En primer lugar, se detallan cada una de las métricas con las cuales se evaluaron los algoritmos involucrados en esta aplicación. Luego de analizar dichas métricas se presenta el algoritmo seleccionado.

Además, se especifican los casos de estudio en los cuales se evaluó la herramienta resultante del proyecto. Los mismos ponen a prueba la funcionalidad implementada en su totalidad.

6.1 Métricas de Algoritmos

En esta sección se mostrarán los resultados de las métricas calculadas sobre cada uno de los tres algoritmos utilizados en esta herramienta.

6.1.1 Precisión

Los resultados de la *precisión* de cada uno de los tres algoritmos se observan en la Figura 6.1.

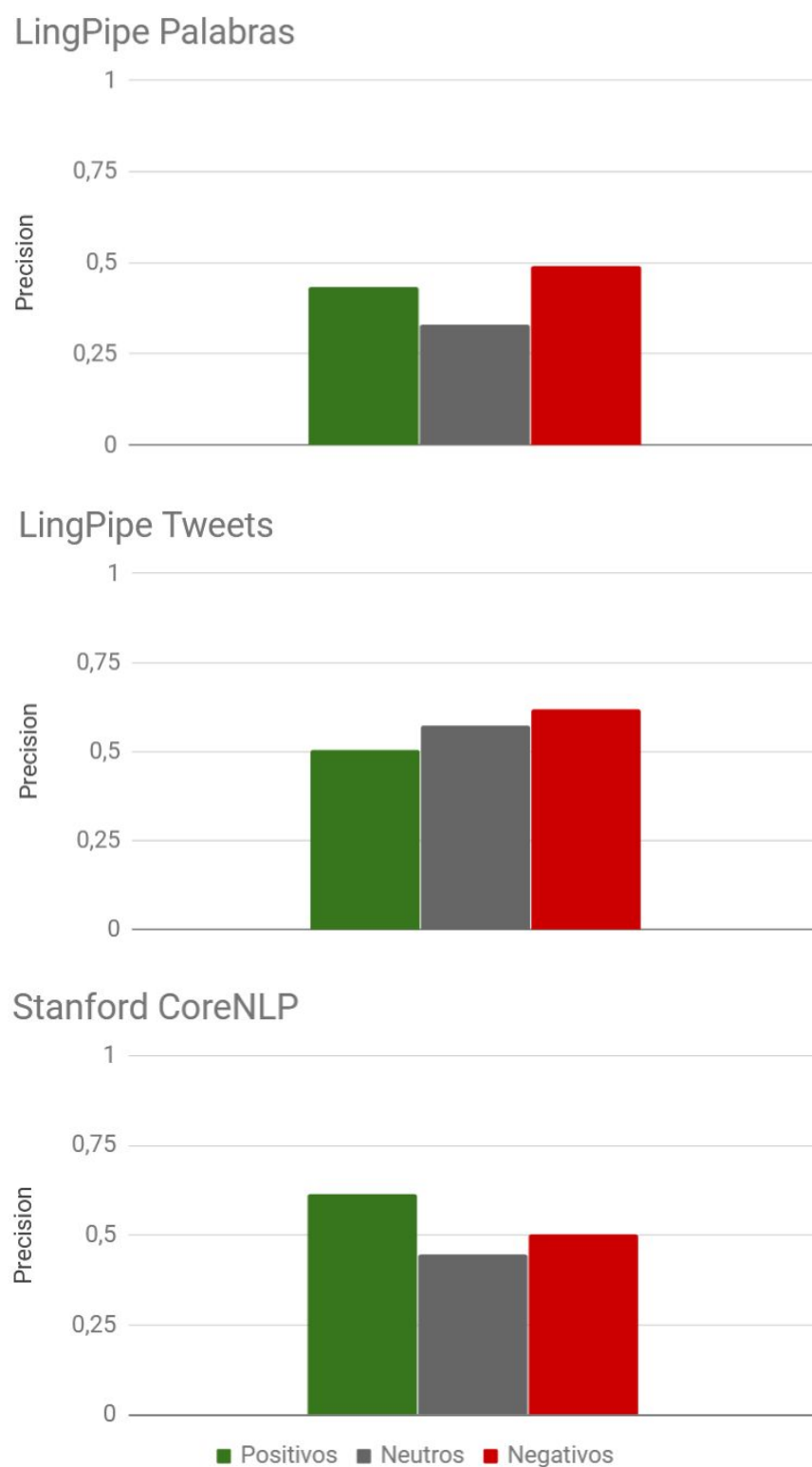


Figura 6.1: Precisión Algoritmos.

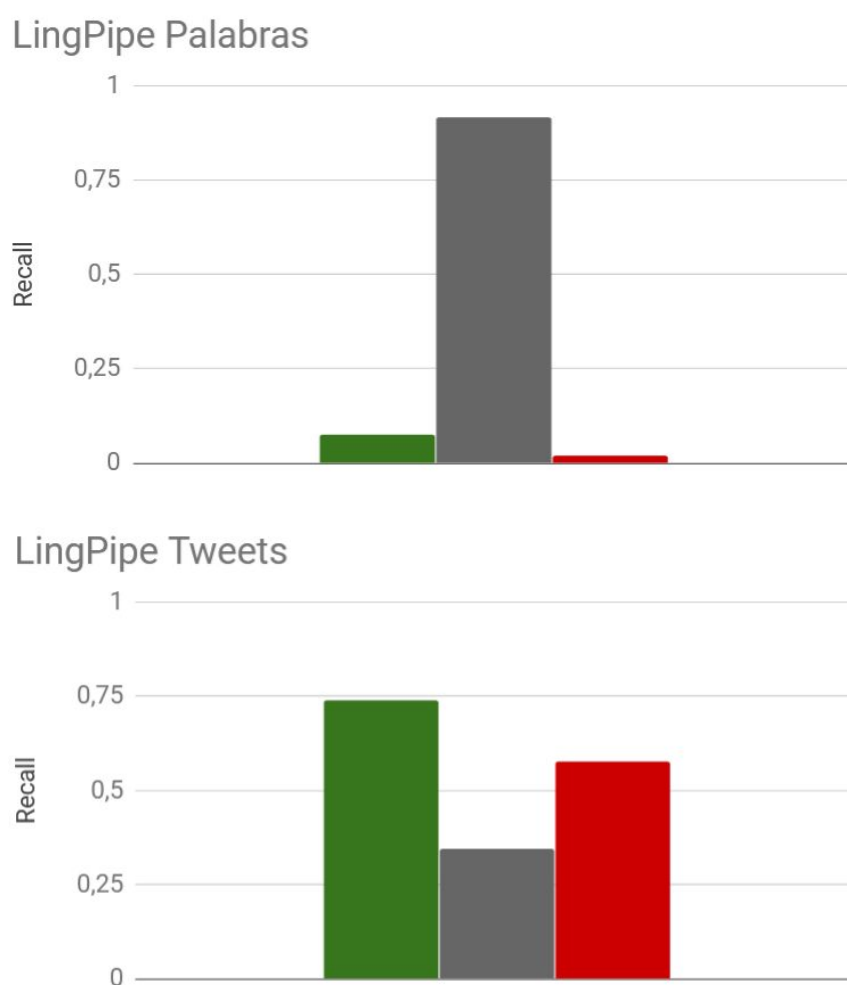
Tal como se aprecia en la Figura 6.1 los resultados de esta métrica son:

- LingPipe Palabras: 43% en positivos. 32% en neutros. 49% en negativos.

- LingPipe Tweets: 50% en positivos. 57% en neutros. 62% en negativos.
- Stanford CoreNLP: 61% en positivos. 44% en neutros. 50% en negativos.

6.1.2 Recall

Los resultados del *recall* de cada uno de los tres algoritmos se observan en la Figura 6.2.



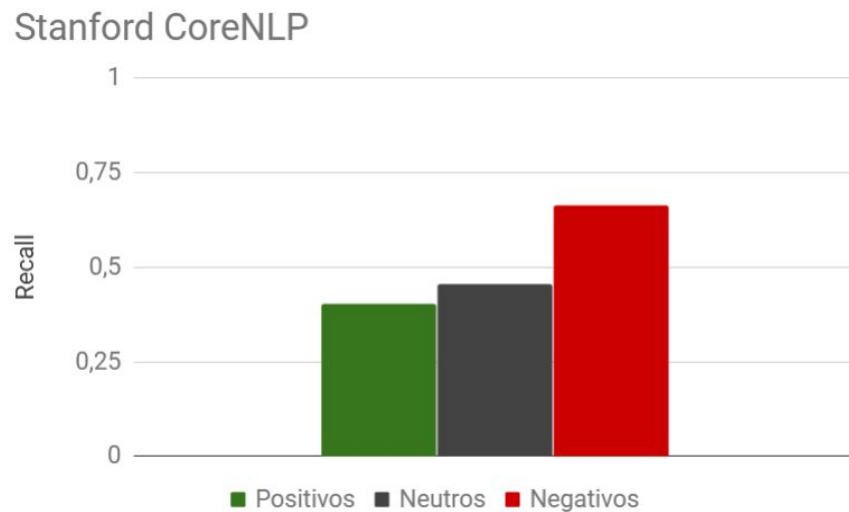


Figura 6.2: Recall Algoritmos.

Tal como se aprecia en la Figura 6.2 los resultados de esta métrica son:

- LingPipe Palabras: 7% en positivos. 91% en neutros. 2% en negativos.
- LingPipe Tweets: 74% en positivos. 34% en neutros. 57% en negativos.
- Stanford CoreNLP: 40% en positivos. 45% en neutros. 66% en negativos.

6.1.3 Performance

Los resultados del tiempo que tardan cada uno de los tres algoritmos en clasificar cinco mil tweets se observan en la Tabla 6.1.

LingPipe Palabras	LingPipe Tweets	Stanford CoreNLP
00' : 10' '	00' : 10' '	9' : 40' '

Tabla 6.1: Performance algoritmos.

6.1.4 Confianza

Los resultados de la distribución de confianza de cada algoritmo, se pueden ver en la Figura 6.3.

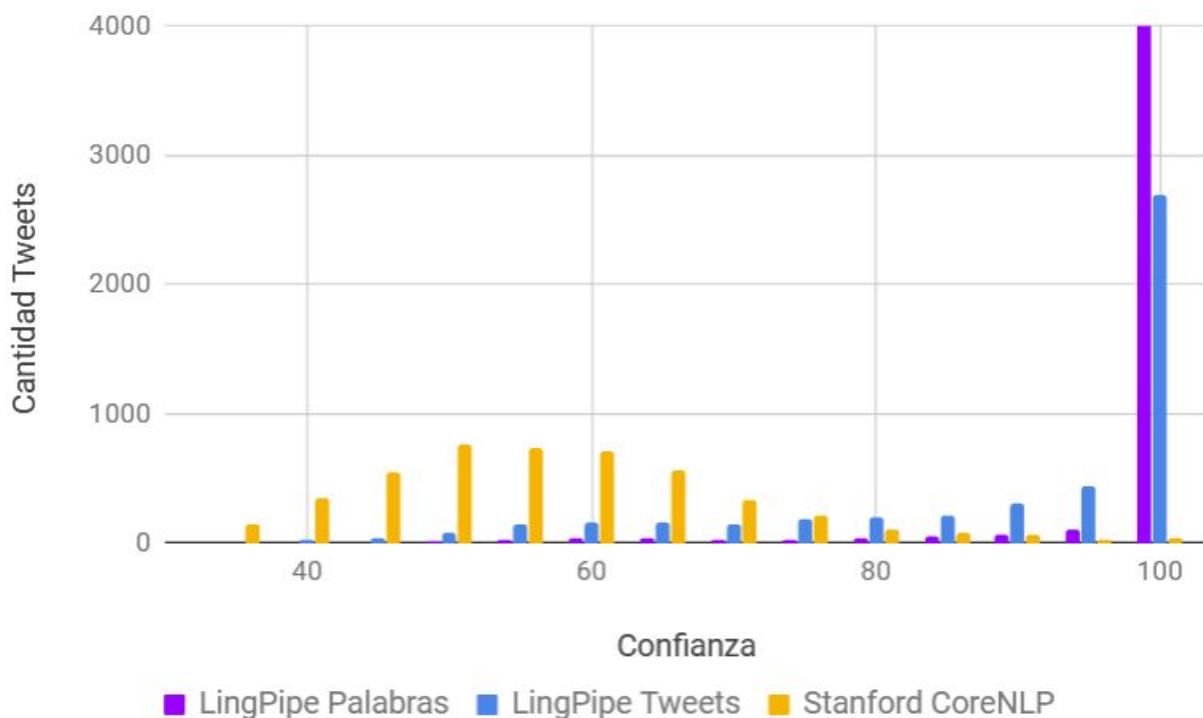


Figura 6.3: Distribución de confianza Algoritmos.

Analizando la Figura 6.3 se puede apreciar que las curvas de LingPipe Palabras y LingPipe Tweets alcanzan sus valores máximos en altos valores de confianza. Específicamente sus puntos máximos son en 99% de confianza con 3034 y 2108 tweets respectivamente. Por otro lado Stanford CoreNLP tiene su máximo en 58% de confianza con solo 168 tweets. Esto se debe a que los valores de confianza que retorna el mismo están mucho más distribuidos, concentrándose mayormente en el rango entre 41% y 66% de confianza.

La segunda métrica calculada fue la de la exactitud por cada valor de confianza. Luego de realizar los cálculos y plasmar los valores obtenidos, el gráfico resultante estaba compuesto por curvas discontinuas. Esto se debe a la gran cantidad de valores

de confianza (0 a 100) y que algunos de dichos valores tienen muy pocos datos. Para solucionarlo, se tomó la decisión de acotar la precisión con la que se muestra la confianza agrupando los valores de a cinco tal como se ve en la Figura 6.4. La misma muestra a partir del valor 30 en el eje X, ya que los valores de confianza anteriores son cero (la confianza retornada mínima es del 33,33%).

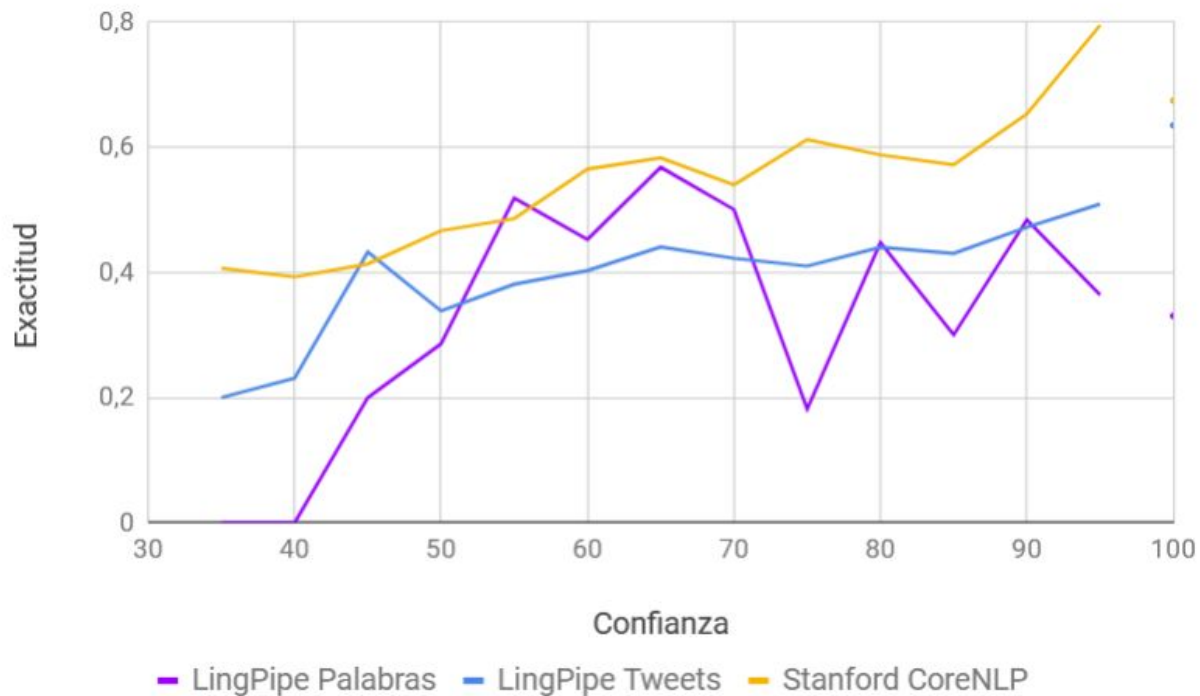


Figura 6.4: Exactitud para cada valor de confianza.

La Figura 6.4 nos muestra que el algoritmo de Stanford CoreNLP es el que muestra mayor exactitud para casi todos los valores de confianza. Por otra parte, en la Figura 6.3 vista anteriormente, se ve que este algoritmo casi nunca devuelve valores mayores al 80%.

Los dos algoritmos restantes retornan valores altos de confianza, y según la Figura 6.4 es LingPipe Tweets el que tiene mejor exactitud en los mismos.

6.1.5 Análisis de resultados

Uno de los algoritmos fue eliminado automáticamente de entre las posibilidades luego de haber sido evaluado con estas métricas. El mismo es LingPipe entrenado con palabras. Esto se debe a que la sensibilidad (recall) de este algoritmo es casi nula con

tweets positivos y negativos. Puntualmente, solo el 7% de tweets positivos y 2% de los tweets negativos son clasificados correctamente.

El resumen de las métricas calculadas para los dos algoritmos restantes se puede visualizar en la Tabla 6.2.

Métrica	LingPipe Tweets	Stanford CoreNLP
Precisión Positivos	0.50	0.61
Precisión Neutros	0.57	0.44
Precisión Negativos	0.62	0.50
Recall Positivos	0.74	0.40
Recall Neutros	0.34	0.45
Recall Negativos	0.57	0.66
Total	3.34	3.06

Tabla 6.2: Tabla comparativa Precision y Recall.

Notar que si bien Stanford CoreNLP tiene mejor *recall* en neutros y negativos, la suma de la diferencia en estas categorías es menor a la diferencia que tiene LingPipe Tweets a su favor en los positivos.

El resumen de las métricas calculadas en base a la confianza para los dos algoritmos restantes se puede visualizar en la Tabla 6.3.

Métrica	LingPipe Tweets	Stanford CoreNLP
Valores de Confianza retornados mayormente	Entre 96 y 99	Entre 41 y 66
Exactitud en valores de confianza entre 96 y 100	0,6344162799	0,6739130435

Tabla 6.3: Tabla comparativa confianza.

Notar que Stanford CoreNLP tiene mayor porcentaje de aciertos en altos valores de confianza, pero los valores retornados mayormente por el mismo no son altos.

Además, luego de realizar los cálculos de la performance no quedaron dudas de que el algoritmo seleccionado para evaluar los casos de estudio debe ser LingPipe Tweets. Tal como se demostró, el mismo obtuvo mejores resultados en las métricas de precisión, recall, confianza y devuelve los resultados en tan solo segundos.

6.2 Caso de estudio 1: RespuestasTweet

En este caso de estudio se evaluará la funcionalidad de la aplicación encargada de analizar las respuestas de los usuarios de Twitter, a un tweet específico. Para comprender mejor los resultados se probó la herramienta con dos usuarios.

6.2.1 Usuario estudiado 1: Demi Lovato

Demi Lovato es una famosa actriz, cantante y compositora estadounidense y es la fundadora junto con Nick Jonas del sello discográfico Safehouse Records. La misma es un usuario muy activo en Twitter y tiene 57,3 millones de seguidores.

El tweet del cual se obtuvieron las respuestas para analizar fue publicado el 27 de junio de 2018 a las 12:18. El texto del mismo es el siguiente:

“To my fans, my family, and my supporters, the ones who never left me, you are my light. I am forever grateful”

“Para mis fans, mi familia y mis seguidores, los que nunca me abandonaron, ustedes son mi luz. Estoy eternamente agradecida”

Los resultados obtenidos son los que se ven en la Figura 6.5.

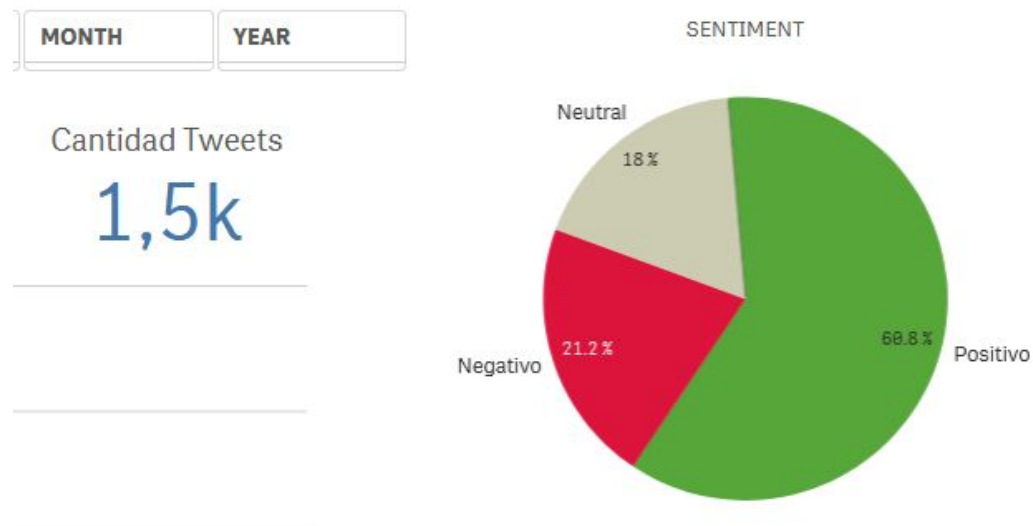


Figura 6.5: Resultado de la clasificación de respuestas Usuario 1.

Tal como se ve en la figura anterior el 60,8% de las respuestas son positivas, el 18% son neutrales, y el 21,2% restante son negativas.

A pesar de significar sólo un 21,2%, se filtraron los resultados con el fin de comprender porque esos usuarios respondieron de forma negativa a dicho tweet.

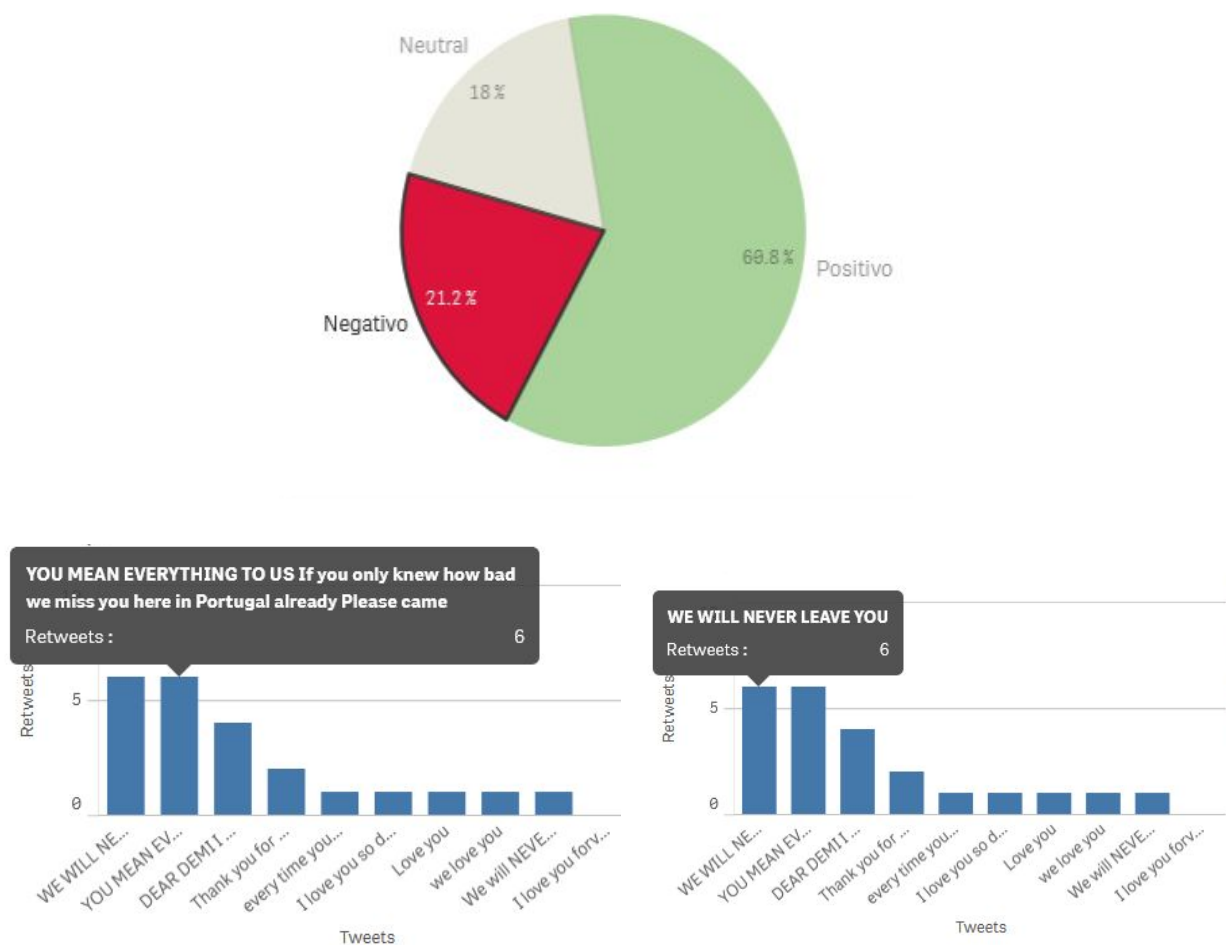


Figura 6.6: Respuestas negativas Usuario 1.

Tal como se lee en la figura Figura 6.6 dichos tweets son falsos positivos. A decir verdad, son tweets positivos que el algoritmo de la herramienta clasificó de manera incorrecta. Las traducciones de los tweets que se visualizan en dicha imagen son:

“USTED SIGNIFICA TODO PARA NOSOTROS. Si solo supieras lo mucho que te extrañamos aquí en Portugal. Por favor regresa ”

“NUNCA TE DEJAREMOS”

6.2.2 Usuario estudiado 2: Donald Trump

El presidente de los estados unidos Donald Trump es un excelente usuario para analizar resultados con esta aplicación. Esto se debe, en principio, a que tiene 52

millones de seguidores. Además, usualmente publica varios tweets por hora todos los días y tiene miles de respuestas en cada uno de ellos.

Se utilizó en este caso un tweet que el presidente publicó el día 18 de mayo de 2018, cerca de las 12:00hs. El texto del tweet es el siguiente:

“School shooting in Texas. Early reports not looking good. God bless all!”

“Disparos escolares en Texas. Los primeros informes no se ven bien. ¡Dios bendiga a todos!”

El tweet hace referencia a la tragedia ocurrida en una escuela secundaria en Texas-EEUU esa misma mañana. Los primeros informes que se dieron a conocer en los medios de comunicación masivos, confirmaban 8 personas asesinadas. Más tarde el informe final confirma 10 personas asesinadas y otras 10 heridas.

Tan solo en la primer hora, el tweet de Trump obtuvo 9400 respuestas de usuarios de todo el mundo. El resultado de los sentimientos analizados en las respuestas es el que se muestra en la Figura 6.7.

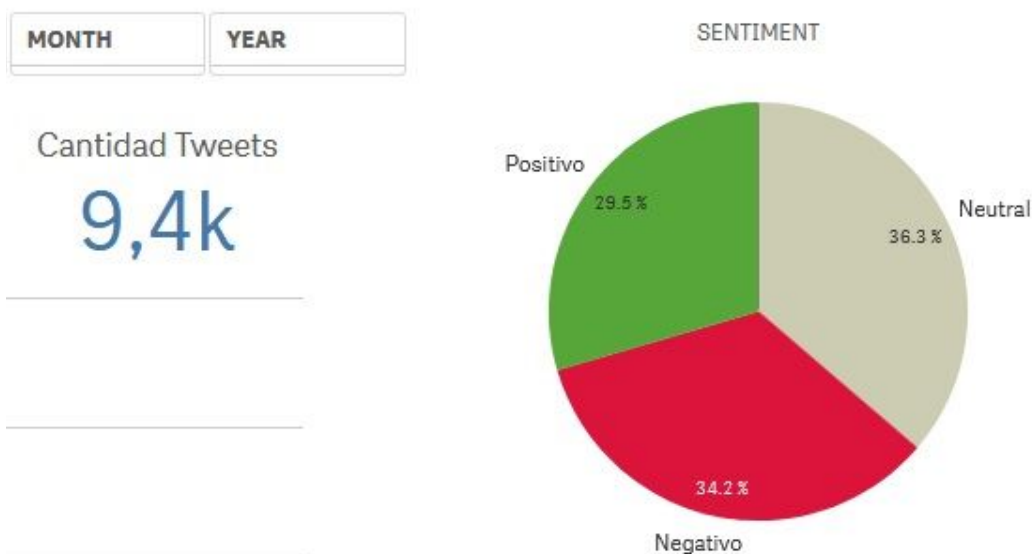


Figura 6.7: Resultado de la clasificación de respuestas Usuario 2.

En la Figura 6.7 se vé que el gráfico está balanceado con un porcentaje muy similar para cada una de las categorías. Los tweets clasificados como positivos conforman el 29.5% del total, los neutrales el 36.3% y los negativos el 34.2%. Este balance en los sentimientos de los tweets es llamativo, dada la naturaleza del hecho reportado, por lo que se analizaron en más detalle los resultados. Para ello, en primer

lugar, fueron filtrados dejando solamente los clasificados como positivos. En la Figura 6.8 se puede ver el mapa con la distribución mundial de los mismos.



Figura 6.8: Resultados filtrados a positivos.

Se seleccionaron algunos tweets específicos de la Figura 6.8 para visualizar su contenido y cómo se relacionaban con el tweet de Trump. Algunos de los tweets clasificados por el algoritmo como positivos son los siguientes:

“U make me laugh..” “Me haces reír”

“ThANks FoR aLL THe "thoughts and prayers"”

“GRACIAS POR TODOS LOS "pensamientos y oraciones"”

“God bless all? WHAT ARE YOU GOING TO DO ABOUT IT?!”

“Dios bendiga a todos? QUÉ VAS A HACER AL RESPECTO?!”

Los ejemplos dejan a la vista que varios de los tweets si bien fueron clasificados como positivos, en realidad son falsos positivos. Uno de los motivos es que la frase “Dios bendiga a todos”, que es una frase positiva, fue utilizada en el tweet de Trump y es referenciada en muchas de las respuestas. Esta frase puede generar que la oración

sea clasificada como positiva, lo cual es incorrecto ya que no es el usuario quien la escribió sino que la cita con ironía.

En la Figura 6.9 se puede ver el mapa con la distribución de tweets clasificados como neutros.



Figura 6.9: Resultados filtrados a neutros.

Algunos de los tweets clasificados por el algoritmo como neutros son los siguientes:

“Time to do something about it!” “Es hora de hacer algo al respecto!”

“That'll help a lot. NOT” “Eso ayudará mucho. NO”

“I know u care” “Sé que te importa”

Leyendo algunos ejemplos en esta categoría, se podría decir que en su mayoría fueron bien clasificados. Esto se debe a que las oraciones contienen en su totalidad vocabulario neutro. Por otra parte, los falsos positivos se deben a la utilización de ironías por parte de los usuarios.

De la misma manera se filtraron tweets negativos tal como se ve en la Figura 6.10.



Figura 6.10: Resultados filtrados a negativos.

Algunos de ellos son:

“ ‘Early reports not looking good’.....well that’s obvious. How could a school shooting look good in any way?”

“ ‘Los primeros informes no se ven bien’ ... bueno, eso es obvio. ¿Cómo podría un tiroteo en la escuela verse bien de alguna manera?”

“We need gun control or you impeached on or the other you choose”

“Necesitamos control de armas o usted acusado lo que usted elija”

Luego de una revisión de los tweets clasificados como negativos, se podría decir que la mayoría de ellos fueron clasificados correctamente debido a la gran cantidad de palabras negativas que contienen. Algunas de ellas son “armas”, “tiroteo”, “matar”, “acusar”, entre otras.

6.3 Caso de estudio 2: Hashtag

En este caso de estudio se evaluará la funcionalidad de la aplicación que se encarga de analizar tweets que contienen una palabra clave específica.

Se encontró un hashtag bastante popular entre los más utilizados de Twitter en el momento. El hashtag es “#TrumpMustGo” y se utiliza principalmente en publicaciones de usuarios que no comparten los ideales del gobierno de los EEUU al mando del presidente Donald Trump. Nuevamente se eligió a este usuario en particular debido únicamente al gran tráfico de tweets que se produce a su alrededor.

La consulta realizada en la aplicación fue la mostrada en la Figura 6.11.

Analizar tweets con cierto Hashtag:

Hashtag:

Cantidad a descargar:

Figura 6.11: Obtener Hashtag.

Los tweets obtenidos efectivamente son 3900, que son la cantidad de tweets publicados en los últimos 10 días que contienen el hashtag dado. Además se puede visualizar, en la Figura 6.12, los resultados de la clasificación del sentimiento de los mismos en la aplicación creada: el 19.4% de los tweets son positivos, el 18.6% son neutros y el 62% son negativos.

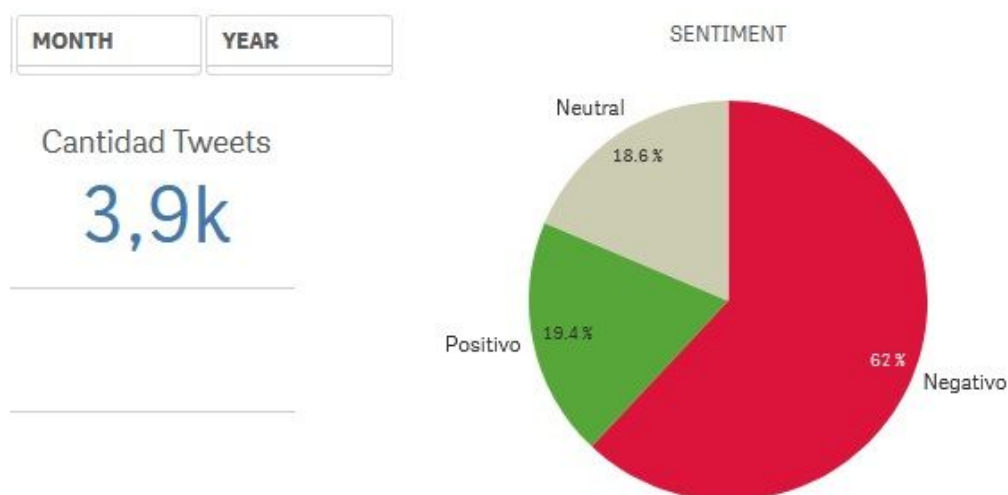


Figura 6.12: Resultado de Clasificar tweets con hashtag.

Otro de los gráficos que se visualiza en la Figura 6.13, nos da la suma de retweets (veces que fue compartido por terceros) para cada categoría. Los neutrales suman un total de 48 mil, los positivos 25 mil mientras que los negativos en mayor medida fueron retuiteados 1 millón 200 mil veces.

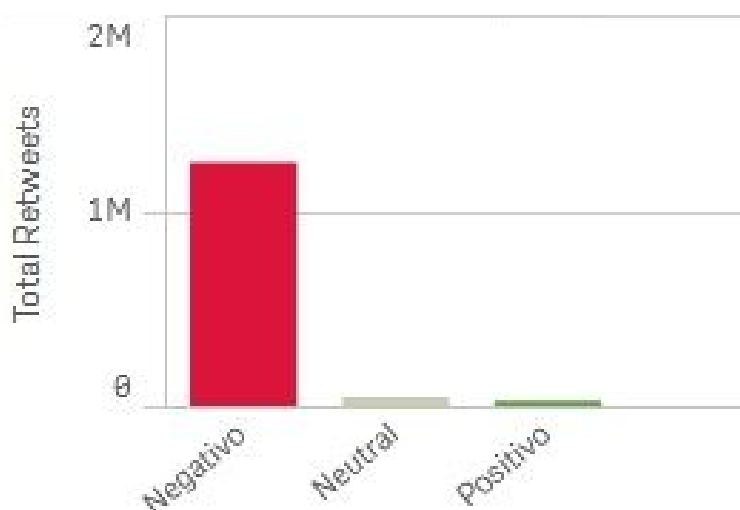


Figura 6.13: Suma de retweets para cada categoría.

El total de retweets negativos se debe en gran medida a un tweet en especial. Este tweet es el que encabeza el ranking de tweets de la aplicación como el más retuiteado entre todos los obtenidos. El texto del mismo es:

*“I’m on Twitter because I’m surrounded by FoxNews zombies. This is my sane place. I’d like to connect with more people who believe the #TrumpCrimeFamily should be removed from the WH
#TrumpMustGo”*

*“Estoy en Twitter porque estoy rodeado de zombies de FoxNews. Este es mi lugar cuerdo. Me gustaría conectarme con más personas que creen que la #TrumpCrimeFamily debería abandonar la Casa Blanca
#TrumpMustGo”*

Tal como se aprecia en la Figura 6.14 este tweet cuenta con casi 1 millón de retweets, lo cual supera en gran medida al resto de los tweets en el ranking.

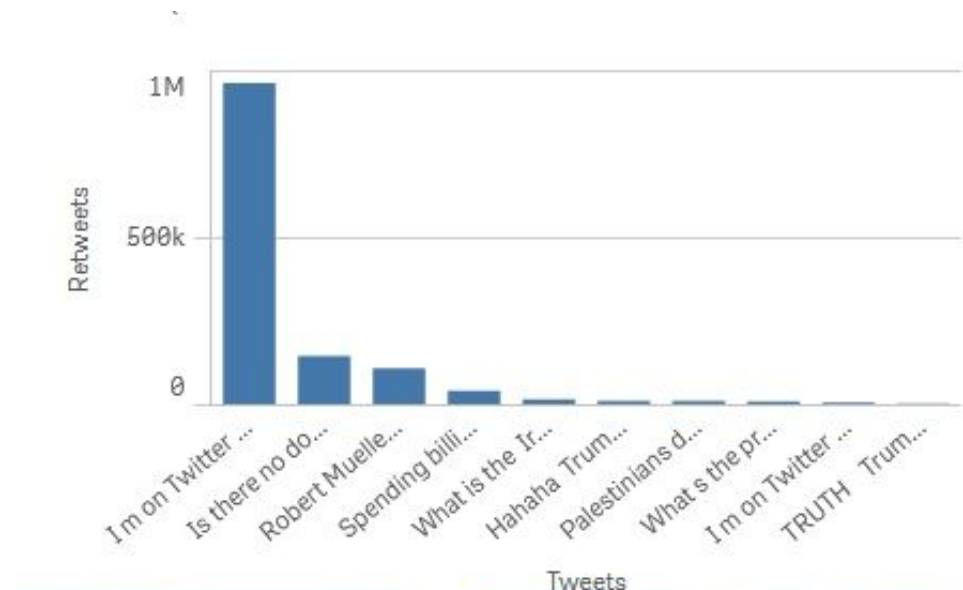


Figura 6.14: Tweet más retwiteado con #TrumpMustGo.

6.4 Caso de estudio 3: TweetsUsuario

En este caso de estudio se evaluará la funcionalidad de la aplicación que se encarga de analizar tweets publicados por un usuario específico.

Para este caso en particular, se decidió analizar y comparar los resultados de la aplicación en diversas empresas reconocidas a nivel mundial. Entre ellas están Toshiba, Netflix, McDonald's, RedBull, Audi, BMW, Starbucks, Coca Cola, PlayStation, GoPro, Porsche, Hp, Google, Youtube, Nintendo, Xbox, Disney, Android, Ubuntu, Sony, Nike, Adidas. Todas estas marcas realizan diversas publicaciones en sus perfiles de Twitter. Algunas de ellas con el fin de difundir nuevos productos o compartir contenido relevante, y otras simplemente para comunicar diferentes publicidades que incluyen la marca.

Se analizaron cerca de 3250 tweets publicados por cada una de ellas. Debido a que las tendencias en los sentimientos de las publicaciones son similares, se tomaron tres casos representativos y se muestran los resultados en en la Figura 6.15.

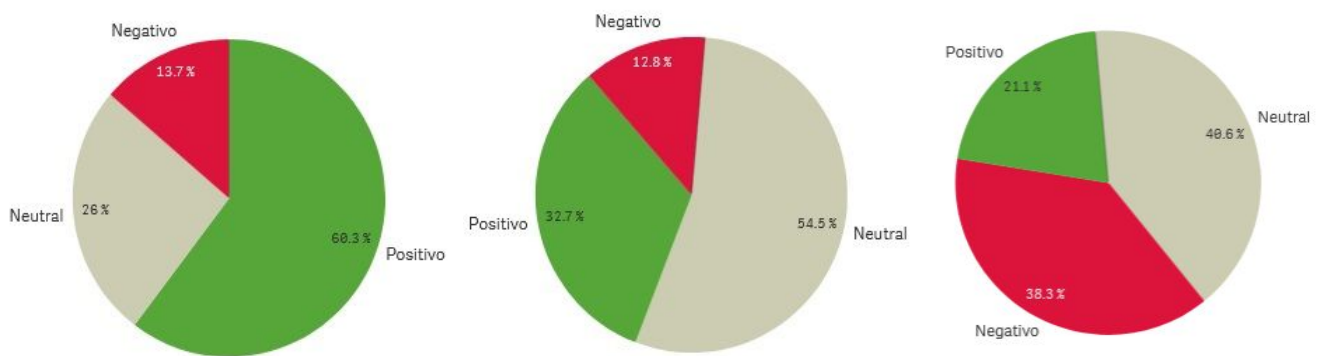


Figura 6.15: Resultado de Clasificar tweets de McDonald's, Coca Cola y Ubuntu.

Tal como se aprecia en la Figura anterior los resultados de las publicaciones son:

- McDonald's: 60,3% positivas. 26% neutras. 13,7% negativas.
- Coca Cola: 32,7% positivas. 54,5% neutras. 12,8% negativas.
- Ubuntu: 21,1% positivas. 40,6% neutras. 38,3% negativas.

Tanto McDonald's como Coca Cola, poseen un porcentaje muy bajo de publicaciones con sentimientos negativos. Por otro lado, Ubuntu tiene un 38,3% de las mismas. Podría deducirse que, Mc'Donalds's y Coca Cola buscan difundir sus productos e insertarlos en la sociedad de la mejor manera posible. Ubuntu, al ser una empresa de software libre, no tiene los mismos intereses.

La cantidad de veces que los usuarios retuitean las publicaciones de cada categoría, se puede observar en la Figura 6.16.

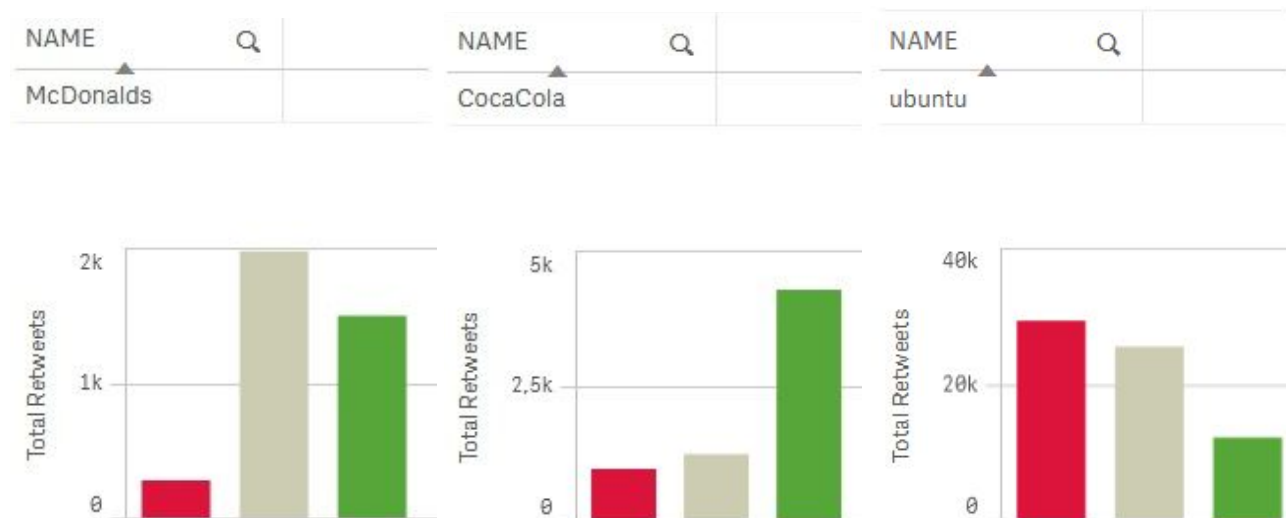


Figura 6.16: Total de retweets para McDonald's, Coca Cola y Ubuntu.

Tal como se observa en la Figura 6.16, McDonald's y Coca Cola no tienen gran número de retweets. Coca Cola suma 6700 retweets en 3500 publicaciones, es decir en promedio republican solo dos veces cada publicación. McDonald's suma aún menos retweets, 3800 en 3500 publicaciones aproximadamente. Por otra parte, Ubuntu acumula en total 66780 retweets, esto es 9,7 veces más que Coca Cola y 17,8 veces más que Mc'Donalds. Esto indicaría que las publicaciones de Ubuntu son de mayor interés para sus seguidores.

Capítulo 7

Conclusiones

En este trabajo final se desarrolló una aplicación web utilizada para analizar los sentimientos en la red social Twitter. La misma, puede ser utilizada para el entretenimiento del usuario ó como una herramienta de negocio, ya que ésta obtiene datos reales de la red social Twitter relacionados a todo tipo de usuarios o temas de discusión. Por ejemplo, un político puede utilizar esta aplicación para investigar si los usuarios de la red social apoyan o no su campaña, o una estrella de rock puede utilizarla para saber qué opinan sus seguidores de su nuevo hit musical. Gracias a ella los usuarios se ahorran el tiempo y el esfuerzo que conlleva leer cada tweet e ir contabilizando los datos. Luego de una investigación de opciones similares disponibles, no se encontró en la web una herramienta con todas las características de la realizada en este proyecto.

Como toda aplicación web, esta herramienta sigue el Modelo Cliente-Servidor. A su vez, del lado del servidor, se integraron dos componentes externos. Uno de ellos es Twitter, quien provee la información de dicha red social mediante su API. Por otro lado se utilizó Qlik Sense, una herramienta de BI robusta para analizar los resultados obtenidos.

La arquitectura sobre la cual se implementó esta aplicación se caracteriza por brindar una alta modificabilidad y escalabilidad. Esto permite incorporar fácilmente nuevos algoritmos o fuentes de datos para analizar. En conjunto, se logró una herramienta de análisis de sentimientos completa e intuitiva.

La herramienta desarrollada tiene la capacidad de reconocer las opiniones de los usuarios en todo tipo de publicaciones en el idioma inglés. El texto de estas publicaciones es preprocesado con el fin de eliminar todos los elementos que no sean de utilidad para la predicción del sentimiento. Además, hace uso de algoritmos

entrenados con técnicas de machine learning, los cuales clasifican los textos de los tweets en positivos, neutros o negativos.

Se utilizaron dos bibliotecas de clasificación, una de las mismas fue entrenada con dos bases de datos diferentes. En otras palabras se implementaron tres algoritmos de clasificación de sentimientos. Se comparó cada uno de ellos con distintas métricas: precisión, sensibilidad y confianza. Luego de dicho análisis, se optó por el algoritmo de la biblioteca LingPipe entrenado con una base de datos de Tweets. El mismo fue utilizado para evaluar la herramienta en los casos de estudio.

A su vez, luego de analizar uno por uno los tweets indicados por el usuario, la aplicación muestra los resultados mediante una herramienta de Inteligencia de Negocio (BI). Esta herramienta permite crear, integrar e incorporar analítica visual atractiva y elegante para cualquier dispositivo o app, desde cualquier sitio y en una sola plataforma. Ofrece una completa cartera de soluciones que proporcionan analítica avanzada a todo el abanico de necesidades dentro de la BI. Gracias a esto, el usuario puede de manera intuitiva, sencilla y rápida explorar y realizar subconsultas sobre los datos que está visualizando. El funcionamiento de la aplicación puede visualizarse en el Anexo 2 : Aplicación Web.

Por último, se evaluó la misma en diversos casos de estudio:

- Respuestas a un tweet: Se utilizaron tweets de usuarios con mas de 50 millones de seguidores.
- Tweets de un usuario: Se analizaron publicaciones de empresas reconocidas a nivel mundial.
- Hashtag: Se buscaron *trending topics* (temas del momento) y se analizaron publicaciones de usuarios referidas a los mismos.

Se comprobó que esta herramienta es muy útil a la hora evaluar la aceptación de una persona o empresa por parte de los usuarios de Twitter. Basta con seleccionar cualquier publicación o usuario, para que la aplicación muestre los sentimientos que generan en dicha comunidad.

Además, permite navegar minuciosamente los datos de salida con el fin de comprender un poco mejor los motivos por los cuales las publicaciones de los usuarios se inclinan hacia un sentimiento en particular. Asimismo esta aplicación es

de gran ayuda a la hora de analizar tendencias relacionadas al consumo de un producto, un acontecimiento, un debate, etc.

7.1 Limitaciones y trabajos futuros

Si bien la aplicación ha demostrado su utilidad analizando sentimientos en Twitter, su versión actual presenta algunas limitaciones. Una de ellas es que la exactitud del algoritmo con el cual se clasifican los sentimientos en tres categorías es del 55%. Cabe destacar, tal como se explicó en el Marco Teórico, que esta clasificación es una tarea difícil incluso para dos personas distintas. Según la investigación de [33] la concordancia humana está entre el 70% y 79%. Si bien la precisión obtenida impide utilizar esta herramienta para tomar decisiones de negocio reales, es una buena base para dar un primer vistazo general de las opiniones y luego analizar los resultados obtenidos en mayor profundidad. Una línea de trabajo futuro consiste en investigar un algoritmo o heurística que mejore la precisión de la clasificación. En principio se podría entrenar el algoritmo Stanford CoreNLP con la base de datos utilizada para entrenar el algoritmo LingPipe, ya que este último obtuvo mejores resultados. Por otro lado, se podrían investigar heurísticas que utilicen la confianza retornada para mejorar la salida. También se podría investigar si los emoticones aportan algún tipo de sentimiento en las publicaciones, permitiendo incluir detección y análisis de los mismos.

También podrían incorporarse algoritmos que detecten a qué hacen referencia los tweets de respuesta de los usuarios. Una de las limitaciones de esta herramienta es que el sentimiento encontrado puede no hacer referencia al tweet al que están respondiendo. Por ejemplo dos empresas A y B son competencia, y un usuario que está a favor de A, hace un comentario diciendo algo negativo de B. Este tweet es clasificado como negativo, está en respuesta a A, pero no necesariamente va dirigido a A.

Otro aspecto que puede mejorarse en esta aplicación es la veracidad de los usuarios de los cuales se obtienen tweets para analizar. Hoy en día existen un gran número de cuentas automatizadas (bots) o personas que intentan trastornar una comunidad (trolls), escribiendo mensajes sarcásticos, incendiarios, polémicos con el fin de lograr que los usuarios se enfrenten en una pelea o tomen una posición en

contra o a favor de algo [32]. Un futuro trabajo es incorporar la detección de bots y trolls para mejorar la calidad de los resultados de la herramienta.

Otra de las limitaciones es que la aplicación está ligada a la red social Twitter. Si bien la arquitectura de la misma permite intercambiar las fuentes de datos, esta versión solo trabaja con información de Twitter. Un futuro trabajo es la integración de esta herramienta con otras redes sociales como Facebook e Instagram, permitiendo al usuario analizar sentimientos de un mayor porcentaje de usuarios.

Bibliografía

[1] John Scott, "Social Network Analysis", 4th edition 2017.

[2] Dave Chaffey, Global Social Media Statistics Summary, 2016.

[3] Kuan-Yu Lin, Hsi-Peng Lu, "Why people use social networking sites: An empirical study integrating network externalities and motivation theory", Department of Information Management, National Taiwan University of Science and Technology.

[4] Petter Bae Brandtzæg, Jan Heim, "Why People Use Social Networking Sites", in Third International Conference, OCSC 2009, Held as Part of HCI International 2009, San Diego, CA, USA.

[5] Tahir M. Nisar, Caroline Whitehead, "Brand interactions and social media: Enhancing user loyalty through social networking sites", Southampton Business School, University of Southampton, UK.

[6] Olivier Serrat, "Social Network Analysis", in "Knowledge Solutions", 2017.

[7] Bing Liu, "Sentiment Analysis and Opinion Mining", in "Synthesis Lectures on Human Language Technologies", University of Illinois at Chicago.

[8] SemEval-2017 Task 4: Sentiment Analysis in Twitter. Sara Rosenthal, Noura Farra, Preslav Nakov. Qatar Computing Research Institute, Hamad bin Khalifa University, Qatar Department of Computer Science, Columbia University IBM Research, USA.

- [9] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In EMNLP 2013, pages 1631–1642.
- [10] Valente TW, "Network models of the diffusion of innovations", in Quantitative Methods in Communication. Cresskill, New Jersey, Hampton Press.
- [11] Farhan Hassan Khan, Saba Bashir, Usman Qamar, "TOM: Twitter opinion mining framework using hybrid classification scheme", in Decision Support Systems.
- [12] Statista dossier: Social media usage worldwide.
- [13] Alan R. Peslak, Penn State University, "Sentiment Analysis and Opinion Mining: Current State of the Art and Review of Google and Yahoo Search Engines Privacy Policies" in JISAR. December 2017.
- [14] Haewoon Kwak, Changhyun Lee, Hosung Park, Sue Moon, "What is Twitter, a Social Network or a News Media?". Department of Computer Science, KAIST.
- [15] Kevin Makice. "Twitter API: Up and running. 2009.
- [16] Bob Carpenter, Breck Baldwin. "Text Analysis with LingPipe 4". LingPipe Publishing. New York 2011.
- [17] Jorge Bernardino, Marco Tereso, "Computational Intelligence and Decision Making - Trends and Applications".
- [18] Haroon Shakirat Oluwatosin, "Client-Server Model". In IOSR Journal of Computer Engineering (IOSR-JCE). School of Computing University Utara Malaysia Kedah.
- [19] G.Vinodhini, RM.Chandrasekar. "Sentiment Analysis and Opinion Mining: A Survey" in 128X International Journal of Advanced Research in Computer Science and Software Engineering.
- [20] "Sentiment Analysis: Emotion in Text". Open datasets
<https://www.figure-eight.com/data-for-everyone/>.
- [21] Stanford CoreNLP. Página oficial. <https://stanfordnlp.github.io/CoreNLP/>.
- [22] LingPipe. Página oficial. <http://alias-i.com/lingpipe/>.

- [23] Manning, Christopher D., Mihai Surdeanu, John Bauer, Jenny Finkel, Steven J. Bethard, and David McClosky. 2014. The Stanford CoreNLP Natural Language Processing Toolkit In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pp. 55-60.
- [24] Richard Socher, Alex Perelygin, Jean Y. Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng and Christopher Potts. "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank". Stanford University, Stanford, CA 94305, USA.
- [25] Roy Thomas Fielding. "Architectural Styles and the Design of Network-based Software Architectures".2000.
- [26] Fette, I. y Melnikov, A. (2011). "The WebSocket Protocol". Internet Engineering Task Force. Consulta: 28 de octubre de 2015.
- [27] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. "Design Patterns, Elements of Reusable Object-Oriented Software". Addison-Wesley. Massachussets, 1994.
- [28] Rita L. Sallam, Cindi Howson, Carlie J. Idoine, Thomas W. Oestreich, James Laurence Richardson, Joao Tapadinhas. "Magic Quadrant for Business Intelligence and Analytics Platforms". February 2017.
- [29] Jussi-Pekka Erkkilä. "WebSocket Security Analysis".Aalto University School of Science
- [30] Compatibilidad de los navegadores.
<https://developer.mozilla.org/es/docs/WebSockets>
<https://caniuse.com/#feat=websockets>.
- [31] Ian Fette. "The WebSocket protocol".Internet-Draft. October 2010.
- [32] María Elena Meneses. Entre Bots y Trolls. "Redes y medios en el proceso electoral 2012: Mismas prácticas, herramientas distintas".Revista Mexicana de Comunicación.
- [33] Peter Halling Hilborg, Emil Blædel Nygaard. "Viability of sentiment analysis in business. Evaluating accuracy and the supporting NLP technologies". 2015.
- [34] Jim Webber, Savas Parastatidis, Ian Robinson. "REST in Practice - Hypermedia and Systems Architecture".

[35] Qlik Sense. Qlik Technologies, Inc. <https://www.qlik.com/us/products/qlik-sense>.

[36] Qlik Sense Documentation. <https://help.qlik.com/>.

[37] Log4j. Apache Software Foundation. <https://logging.apache.org/log4j/2.x/>.

[38] Twitter4j. Unofficial library. Apache License 2.0. <http://twitter4j.org/en/index.html>.

Anexo 1

Pseudocódigos

Algoritmo 1

```
public int clasificar( tweet )  
//Elementos de Stanford que se utilizaran  
  
Stanford.SetPropiedades="annotators, tokenize, split, parse, sentiment";  
  
//Puede que el tweet no tenga texto, solo emoticones, menciones etc  
  
if (tweet no nulo y su longitud mayor a 0)  
    sentimientoPrincipal= Stanford.clasificarSentimiento(tweet)  
  
devolver sentimientoPrincipal;
```

Algoritmo 2

```
public LingPipe()  
clasificador=(LMClassifier)AbstractExternalizable.readObject(new  
File("classifier.txt"));  
//classifier.txt es el archivo que almacena la configuración del algoritmo entrenado
```



```

clasificador= LingPipe("classifier.txt");
public int clasificar( tweet)
    ConditionalClassification classification = clasificador.classify(tweet);
    //0=categoría mas probable
    confianza=classification.conditionalProbability(0);
    return classification.bestCategory();//devuelve la categoría mas probable

```

Algoritmo 3

```

for ( cada categoria : cat )
    new clasificación(cat); //Creo clasificación para la cat
    get archivos de entrenamiento;//obtengo archivos de datos de entrenamiento
    for (cada archivo : arch)
        datos = Files.readFromFile(archivos);//leo el contenido
        clasificador = train(datos, clasificación);//creo un objeto entrenado a partir de
la clasificación

```

Anexo 2

Aplicación Web

En este anexo se mostrará el funcionamiento de la aplicación creada. En principio, al cargar la página, nos encontramos con las tres posibles fuentes de tweets a utilizar. Para este ejemplo se utilizan “Tweets con cierto Hashtag”, el hashtag ingresado es “Obama” y se especifica que se analicen 5000 tweets.



Analizar respuestas a un tweet:

Nombre usuario:

Identificador del tweet:

Cantidad a descargar:

Analizar

Analizar tweets con cierto Hashtag:

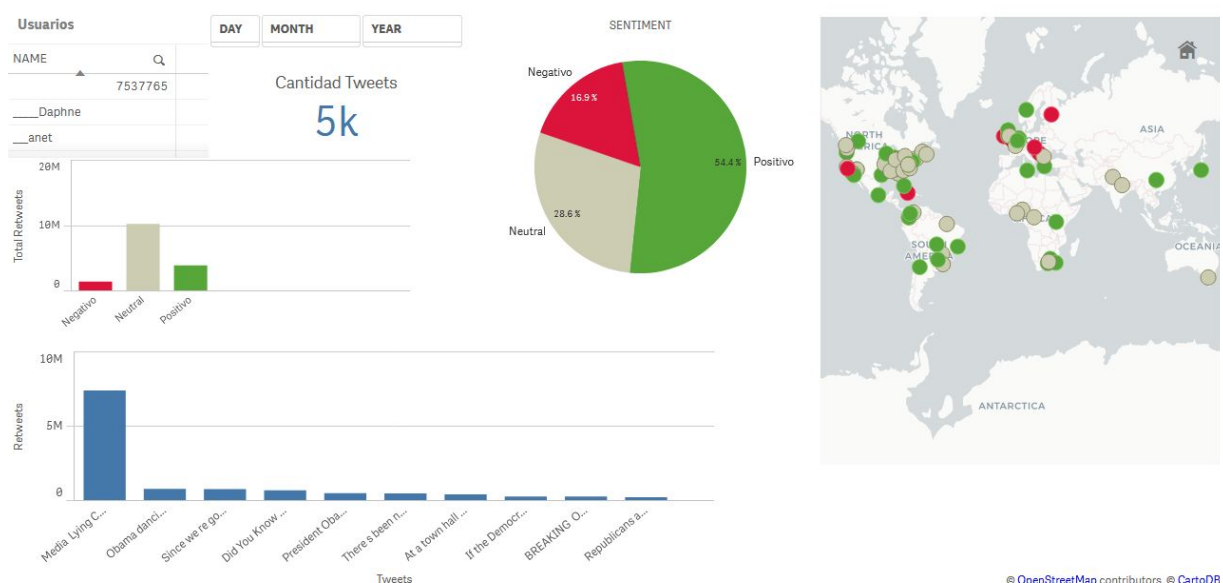
Hashtag:

Cantidad a descargar:

Analizar

No se ha aplicado ninguna selección

Los resultados del análisis de sentimientos de los tweets son mostrados de forma gráfica e incluyen la siguiente información: nombres de usuarios, fecha, sentimiento, cantidad de retweets por categoría, ranking de retweets, ubicación geográfica y cantidad de tweets. Dichos gráficos son los siguientes.

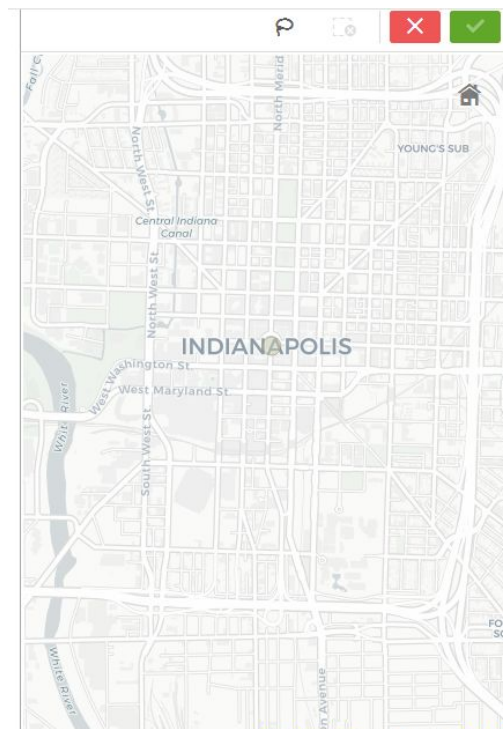


Con el fin de comprender mejor los datos, los mismos pueden ser filtrados en la

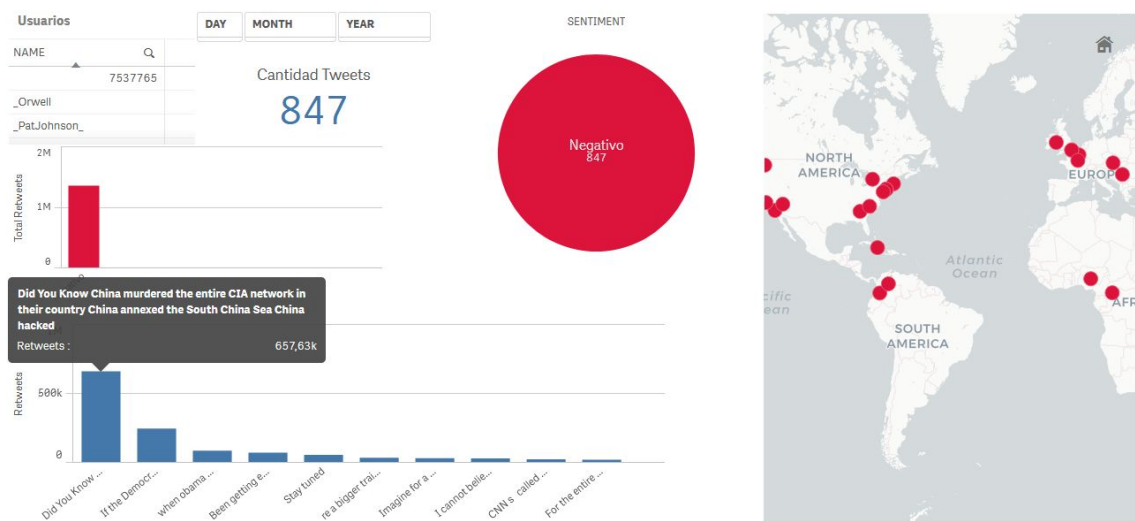
dirección que el usuario desee. Por ejemplo, a continuación se filtran los tweets clasificados como neutros.



En la imagen anterior se puede ver que cada uno de los gráficos fue actualizado con los valores de los tweets neutros, ignorando negativos y positivos. Si hacemos zoom en el mapa, podremos ver la ubicación exacta desde donde el usuario realizó la publicación.

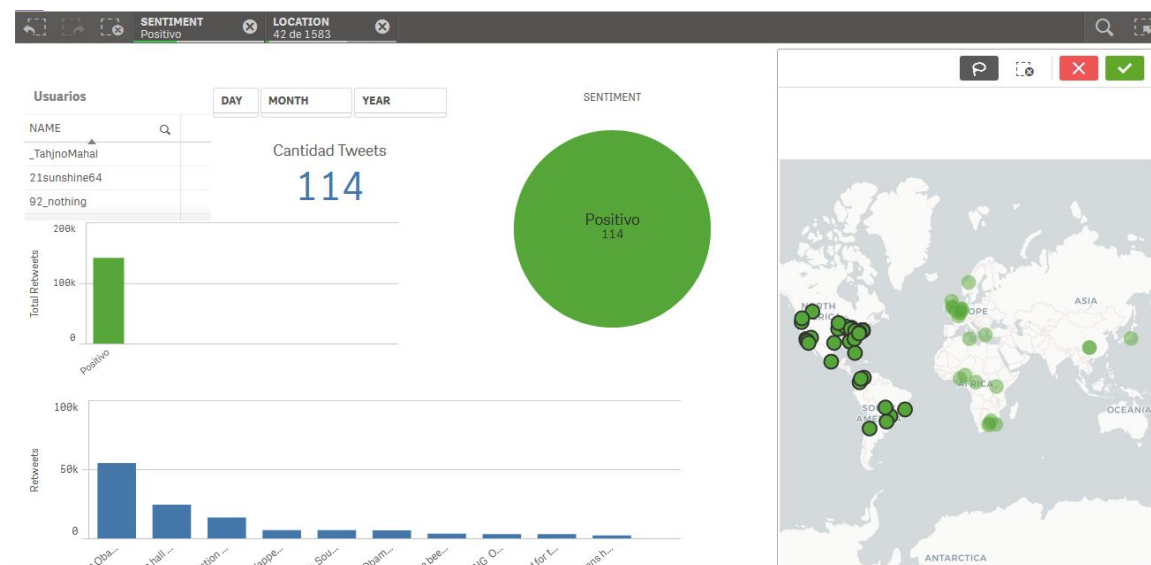


De la misma manera se pueden filtrar los tweets negativos.



A su vez, en la imagen anterior, se posicionó el puntero del mouse sobre el ranking de retweets. El mismo nos muestra de todos los tweets negativos, cuáles son los que tienen mayor número de retweets. La herramienta permite ver el texto de los tweets y los usuarios que lo publicaron.

Por último se filtran los tweets positivos.



En la imagen de los tweets positivos se realizó un filtro doble. En primer lugar se filtraron los tweets positivos. Luego, en el mapa, con la herramienta “lazo” se seleccionaron los tweets del continente americano. Tal como se ve a la izquierda arriba hay dos filtros aplicados a los resultados, y podrían ser más (por fecha, por usuario, etc).